My Project

Gerado por Doxygen 1.16.1

1 Índice dos Componentes 1
1.1 Lista de Classes . . . 1

2 Índice dos Arquivos 3
2.1 Lista de Arquivos . . . 3

3 Classes 5
3.1 Referência da Estrutura Button . . . 5
3.1.1 Descrição detalhada . . . 5
3.1.2 Atributos . . . 5
3.1.2.1 lastDebounceTime . . . 5
3.1.2.2 lastRawReading . . . 6
3.1.2.3 state . . . 6
3.2 Referência da Estrutura ConfigurationData . . . 6
3.2.1 Descrição detalhada . . . 6
3.2.2 Atributos . . . 7
3.2.2.1 mqttDeviceId . . . 7
3.2.2.2 mqttPass . . . 7
3.2.2.3 mqttPort . . . 7
3.2.2.4 mqttServer . . . 7
3.2.2.5 mqttUser . . . 7
3.2.2.6 valid . . . 7
3.2.2.7 wifiPass . . . 7
3.2.2.8 wifiSSID . . . 8
3.3 Referência da Classe List_NursingCall . . . 8
3.3.1 Descrição detalhada . . . 9
3.3.2 Construtores e Destrutores . . . 9
3.3.2.1 List_NursingCall() . . . 9
3.3.2.2 ~List_NursingCall() . . . 9
3.3.3 Documentação das funções . . . 10
3.3.3.1 add() . . . 10
3.3.3.2 clear() . . . 10
3.3.3.3 getDoNotRemoveCurrent() . . . 11
3.3.3.4 getIdCurrent() . . . 11
3.3.3.5 getInfirmaryCurrent() . . . 11
3.3.3.6 getTotal() . . . 11
3.3.3.7 hasNursingCall() . . . 12
3.3.3.8 next() . . . 12
3.3.3.9 prev() . . . 12
3.3.3.10 removalById() . . . 13
3.3.3.11 removeCurrent() . . . 13
3.3.3.12 setDoNotRemoveCurrent() . . . 14
3.4 Referência da Estrutura NursingCall . . . 14

3.4.1 Descrição detalhada . . . 14
3.4.2 Atributos . . . 15
3.4.2.1 id . . . 15
3.4.2.2 infirmary . . . 15
3.4.2.3 next . . . 15
3.4.2.4 prev . . . 15

4 Arquivos **17**
4.1 Referência do Arquivo microcontrolador_central/buttons.cpp . . . 17
4.1.1 Descrição detalhada . . . 18
4.1.2 Funções . . . 19
4.1.2.1 checkButton() . . . 19
4.1.2.2 enableButtonInterruptChange() . . . 19
4.1.2.3 initButtons() . . . 19
4.1.2.4 isConfigurationMode() . . . 20
4.1.2.5 longPressButtonISR() . . . 20
4.1.3 Variáveis . . . 20
4.1.3.1 button_delete . . . 20
4.1.3.2 button_next . . . 20
4.1.3.3 button_prev . . . 21
4.1.3.4 buttonPressStartTime . . . 21
4.1.3.5 DEBOUNCE_DELAY . . . 21
4.1.3.6 isButtonLongPressActivate . . . 21
4.1.3.7 isButtonPressed . . . 21
4.2 Referência do Arquivo microcontrolador_central/buttons.h . . . 21
4.2.1 Descrição detalhada . . . 22
4.2.2 Funções . . . 23
4.2.2.1 checkButton() . . . 23
4.2.2.2 enableButtonInterruptChange() . . . 24
4.2.2.3 initButtons() . . . 24
4.2.2.4 isConfigurationMode() . . . 24
4.2.3 Variáveis . . . 25
4.2.3.1 button_delete . . . 25
4.2.3.2 button_next . . . 25
4.2.3.3 button_prev . . . 25
4.3 buttons.h . . . 25
4.4 Referência do Arquivo microcontrolador_central/buzzer.cpp . . . 26
4.4.1 Descrição detalhada . . . 26
4.4.2 Funções . . . 27
4.4.2.1 buzzerInit() . . . 27
4.4.2.2 doesHaveNotificationBuzzer() . . . 27
4.4.2.3 enableSoundAlert() . . . 27

4.4.2.4 toggleBuzzer() . . . 27
4.4.3 Variáveis . . . 28
4.4.3.1 isBuzzerNotificationEnabled . . . 28
4.4.3.2 lastBuzzerPlayTime . . . 28
4.4.3.3 numberAlertIssued . . . 28
4.5 Referência do Arquivo microcontrolador_central/buzzer.h . . . 28
4.5.1 Descrição detalhada . . . 29
4.5.2 Funções . . . 29
4.5.2.1 buzzerInit() . . . 29
4.5.2.2 doesHaveNotificationBuzzer() . . . 30
4.5.2.3 enableSoundAlert() . . . 30
4.5.2.4 toggleBuzzer() . . . 30
4.5.3 Variáveis . . . 31
4.5.3.1 BUZZER_INTERVAL_MS . . . 31
4.6 buzzer.h . . . 31
4.7 Referência do Arquivo microcontrolador_central/config.h . . . 31
4.7.1 Descrição detalhada . . . 32
4.7.2 Variáveis . . . 33
4.7.2.1 LOG_MODE . . . 33
4.7.2.2 MQTT_PORT . . . 33
4.7.2.3 NUMBER_SOUND_ALERTS . . . 33
4.7.2.4 PASSWORD_ACCESS_POINT . . . 33
4.7.2.5 PIN_BUZZER . . . 34
4.7.2.6 PIN_LED . . . 34
4.7.2.7 SSID_ACCESS_POINT . . . 34
4.8 config.h . . . 34
4.9 Referência do Arquivo microcontrolador_central/config_storage.cpp . . . 35
4.9.1 Descrição detalhada . . . 36
4.9.2 Funções . . . 36
4.9.2.1 hasConfigData() . . . 36
4.9.2.2 initConfigStorage() . . . 37
4.9.2.3 loadConfig() . . . 37
4.9.2.4 saveConfigutionData() . . . 38
4.9.3 Variáveis . . . 38
4.9.3.1 cfg . . . 38
4.9.3.2 CONFIG_PATH . . . 39
4.10 Referência do Arquivo microcontrolador_central/config_storage.h . . . 39
4.10.1 Descrição detalhada . . . 40
4.10.2 Funções . . . 40
4.10.2.1 hasConfigData() . . . 40
4.10.2.2 initConfigStorage() . . . 41
4.10.2.3 loadConfig() . . . 41

4.10.2.4 saveConfigutionData() . . . 42
4.10.3 Variáveis . . . 43
4.10.3.1 cfg . . . 43
4.11 config_storage.h . . . 43
4.12 Referência do Arquivo microcontrolador_central/display_LCD-2004_I2C.cpp . . . 44
4.12.1 Descrição detalhada . . . 45
4.12.2 Funções . . . 45
4.12.2.1 clearLine() . . . 45
4.12.2.2 fixed_data() . . . 45
4.12.2.3 lcd() . . . 46
4.12.2.4 lcd2004_init() . . . 46
4.12.2.5 showExclusionConfirm() . . . 46
4.12.2.6 showFailureMessage() . . . 47
4.12.2.7 showInfirmaryNumber() . . . 47
4.12.2.8 showIPAddress() . . . 48
4.12.3 Variáveis . . . 48
4.12.3.1 currentScreen . . . 48
4.12.3.2 I2C_ADDR . . . 48
4.13 Referência do Arquivo microcontrolador_central/display_LCD-2004_I2C.h . . . 48
4.13.1 Descrição detalhada . . . 49
4.13.2 Enumerações . . . 50
4.13.2.1 DisplayState . . . 50
4.13.3 Funções . . . 50
4.13.3.1 lcd2004_init() . . . 50
4.13.3.2 showExclusionConfirm() . . . 51
4.13.3.3 showFailureMessage() . . . 51
4.13.3.4 showInfirmaryNumber() . . . 52
4.13.3.5 showIPAddress() . . . 53
4.13.4 Variáveis . . . 53
4.13.4.1 currentScreen . . . 53
4.13.4.2 MESSAGE_MQTT . . . 53
4.13.4.3 MESSAGE_WIFI . . . 53
4.14 display_LCD-2004_I2C.h . . . 54
4.15 Referência do Arquivo microcontrolador_central/DoublyLinkedList_NursingCall.cpp . . . 54
4.15.1 Descrição detalhada . . . 54
4.16 DoublyLinkedList_NursingCall.h . . . 55
4.17 Referência do Arquivo microcontrolador_central/jsonDataProcessing.cpp . . . 56
4.17.1 Descrição detalhada . . . 57
4.17.2 Funções . . . 57
4.17.2.1 createJsonPayload() . . . 57
4.17.2.2 creteJsonPayloadConfirmationMessage() . . . 58
4.17.2.3 getPayloadID() . . . 58

4.17.2.4 processing_json_MQTT() . . . 59
4.18 Referência do Arquivo microcontrolador_central/jsonDataProcessing.h . . . 59
4.18.1 Descrição detalhada . . . 60
4.18.2 Funções . . . 60
4.18.2.1 createJsonPayload() . . . 60
4.18.2.2 creteJsonPayloadConfirmationMessage() . . . 61
4.18.2.3 getPayloadID() . . . 62
4.18.2.4 processing_json_MQTT() . . . 63
4.19 jsonDataProcessing.h . . . 64
4.20 Referência do Arquivo microcontrolador_central/led.cpp . . . 64
4.20.1 Descrição detalhada . . . 65
4.20.2 Funções . . . 65
4.20.2.1 ledInit() . . . 65
4.20.2.2 toggleLed() . . . 66
4.20.2.3 turnOffLed() . . . 66
4.20.2.4 turnOnLed() . . . 66
4.20.3 Variáveis . . . 66
4.20.3.1 lastTimeLedTurnedOn . . . 66
4.20.3.2 LED_INTERVAL_MS . . . 66
4.21 Referência do Arquivo microcontrolador_central/led.h . . . 67
4.21.1 Descrição detalhada . . . 67
4.21.2 Funções . . . 68
4.21.2.1 ledInit() . . . 68
4.21.2.2 toggleLed() . . . 68
4.21.2.3 turnOffLed() . . . 68
4.21.2.4 turnOnLed() . . . 68
4.22 led.h . . . 69
4.23 Referência do Arquivo microcontrolador_central/listNursingCall_utils.cpp . . . 69
4.23.1 Descrição detalhada . . . 69
4.23.2 Variáveis . . . 70
4.23.2.1 listCalls . . . 70
4.23.2.2 listUpdated . . . 70
4.24 Referência do Arquivo microcontrolador_central/listNursingCall_utils.h . . . 70
4.24.1 Descrição detalhada . . . 71
4.24.2 Variáveis . . . 72
4.24.2.1 listCalls . . . 72
4.24.2.2 listUpdated . . . 72
4.25 listNursingCall_utils.h . . . 73
4.26 Referência do Arquivo microcontrolador_central/log.cpp . . . 73
4.26.1 Descrição detalhada . . . 73
4.26.2 Funções . . . 74
4.26.2.1 log() . . . 74

4.26.2.2 logInit() . . . 74
4.26.3 Variáveis . . . 74
4.26.3.1 CURRENT_LOG_LEVEL . . . 74
4.27 Referência do Arquivo microcontrolador_central/log.h . . . 75
4.27.1 Descrição detalhada . . . 75
4.27.2 Enumerações . . . 75
4.27.2.1 LogLevel . . . 75
4.27.3 Funções . . . 76
4.27.3.1 log() . . . 76
4.27.3.2 logInit() . . . 76
4.27.4 Variáveis . . . 77
4.27.4.1 CURRENT_LOG_LEVEL . . . 77
4.28 log.h . . . 77
4.29 Referência do Arquivo microcontrolador_central/mqtt.cpp . . . 77
4.29.1 Descrição detalhada . . . 78
4.29.2 Funções . . . 79
4.29.2.1 callback() . . . 79
4.29.2.2 checkMQTTConnected() . . . 79
4.29.2.3 client() . . . 80
4.29.2.4 publicReponseDivice() . . . 80
4.29.2.5 resetMQTTClient() . . . 81
4.29.2.6 setupMQTT() . . . 81
4.29.3 Variáveis . . . 81
4.29.3.1 calledToBeErased . . . 81
4.29.3.2 espClient . . . 81
4.29.3.3 hasOKMessage . . . 82
4.30 Referência do Arquivo microcontrolador_central/mqtt.h . . . 82
4.30.1 Descrição detalhada . . . 82
4.30.2 Funções . . . 83
4.30.2.1 callback() . . . 83
4.30.2.2 checkMQTTConnected() . . . 83
4.30.2.3 publicReponseDivice() . . . 84
4.30.2.4 setupMQTT() . . . 84
4.30.3 Variáveis . . . 85
4.30.3.1 calledToBeErased . . . 85
4.30.3.2 client . . . 85
4.30.3.3 hasOKMessage . . . 85
4.31 mqtt.h . . . 86
4.32 Referência do Arquivo microcontrolador_central/server.cpp . . . 86
4.32.1 Descrição detalhada . . . 87
4.32.1.1 Parâmetros configuráveis . . . 87
4.32.1.2 Fluxo de funcionamento . . . 87

4.32.1.3 Dependências . . . 88
4.32.1.4 Arquitetura . . . 88
4.32.2 Funções . . . 88
4.32.2.1 server_handle_loop() . . . 88
4.32.2.2 serverSetupRoutes() . . . 89
4.32.2.3 startServer() . . . 89
4.32.2.4 stopServer() . . . 89
4.32.3 Variáveis . . . 90
4.32.3.1 serverRunning . . . 90
4.33 Referência do Arquivo microcontrolador_central/server.h . . . 90
4.33.1 Descrição detalhada . . . 90
4.33.1.1 Funcionalidades principais . . . 90
4.33.1.2 Fluxo de uso . . . 91
4.33.1.3 Dependências . . . 91
4.33.2 Funções . . . 91
4.33.2.1 server_handle_loop() . . . 91
4.33.2.2 server_setup_routes() . . . 92
4.33.2.3 startServer() . . . 92
4.33.2.4 stopServer() . . . 92
4.34 server.h . . . 93
4.35 Referência do Arquivo microcontrolador_central/wifi_utils.cpp . . . 93
4.35.1 Descrição detalhada . . . 94
4.35.1.1 Fluxo de funcionamento . . . 94
4.35.1.2 Dependências . . . 94
4.35.2 Funções . . . 95
4.35.2.1 checkAndReconnectWifi() . . . 95
4.35.2.2 connectToWiFi() . . . 95
4.35.2.3 createAccessPoint() . . . 95
4.35.2.4 IPparserToConstChar() . . . 96
4.35.2.5 wifiIsConnected() . . . 96
4.35.3 Variáveis . . . 96
4.35.3.1 ipBuffer . . . 96
4.36 Referência do Arquivo microcontrolador_central/wifi_utils.h . . . 96
4.36.1 Descrição detalhada . . . 97
4.36.1.1 Fluxo típico de uso . . . 97
4.36.2 Funções . . . 98
4.36.2.1 checkAndReconnectWifi() . . . 98
4.36.2.2 connectToWiFi() . . . 98
4.36.2.3 createAccessPoint() . . . 99
4.36.2.4 wifiIsConnected() . . . 99
4.37 wifi_utils.h . . . 99

5 Exemplos **101**

5.1 Exemplo 101

Índice Remissivo **103**

Capítulo 1

Índice dos Componentes

1.1 Lista de Classes

Aqui estão as classes, estruturas, uniões e interfaces e suas respectivas descrições:

Button
Estrutura que representa um botão físico do sistema 5
ConfigurationData
Estrutura que armazena os parâmetros de configuração do sistema 6
List_NursingCall
Classe responsável por gerenciar a lista de chamadas de enfermaria 8
NursingCall
Estrutura que representa um nó da lista de chamadas 14

Capítulo 2

Índice dos Arquivos

2.1 Lista de Arquivos

Esta é a lista de todos os arquivos documentados e suas respectivas descrições:

microcontrolador_central/buttons.cpp
Implementação do gerenciamento de botões do sistema 17
microcontrolador_central/buttons.h
Interface do módulo de gerenciamento de botões físicos 21
microcontrolador_central/buzzer.cpp
Implementação do módulo de controle do buzzer 26
microcontrolador_central/buzzer.h
Interface do módulo de controle do buzzer 28
microcontrolador_central/config.h
Arquivo central de configuração do firmware 31
microcontrolador_central/config_storage.cpp
Implementação do módulo de armazenamento de configurações do sistema 35
microcontrolador_central/config_storage.h
Interface do módulo de armazenamento persistente de configuração 39
microcontrolador_central/display_LCD-2004_I2C.cpp
Implementação do módulo de controle do display LCD 20x4 via I2C 44
microcontrolador_central/display_LCD-2004_I2C.h
Interface do módulo de controle do display LCD 20x4 via I2C 48
microcontrolador_central/DoublyLinkedList_NursingCall.cpp
Implementação de uma lista duplamente encadeada para gerenciamento de chamadas de enfermagem 54
microcontrolador_central/DoublyLinkedList_NursingCall.h 55
microcontrolador_central/jsonDataProcessing.cpp
Processamento de mensagens JSON recebidas e enviadas via MQTT 56
microcontrolador_central/jsonDataProcessing.h
Interface para processamento de mensagens JSON utilizadas na comunicação MQTT 59
microcontrolador_central/led.cpp
Implementação do controle do LED indicador do sistema 64
microcontrolador_central/led.h
Interface para controle do LED indicador do sistema 67
microcontrolador_central/listNursingCall_utils.cpp
Definição das variáveis globais utilizadas no gerenciamento da lista de chamadas de enfermaria 69
microcontrolador_central/listNursingCall_utils.h
Declarações globais para gerenciamento da lista de chamadas de enfermaria 70
microcontrolador_central/log.cpp
Implementação de funções de logging para depuração e monitoramento do sistema 73

microcontrolador_central/log.h
Declarações para o sistema de logging do projeto 75
microcontrolador_central/mqtt.cpp
Implementação do módulo de comunicação MQTT do sistema 77
microcontrolador_central/mqtt.h
Interface do módulo de comunicação MQTT do dispositivo 82
microcontrolador_central/server.cpp
Implementação do servidor HTTP para configuração do dispositivo 86
microcontrolador_central/server.h
Interface do módulo de servidor HTTP para configuração do dispositivo 90
microcontrolador_central/wifi_utils.cpp
Implementação do módulo de gerenciamento de conexão Wi-Fi do dispositivo 93
microcontrolador_central/wifi_utils.h
Interface do módulo de gerenciamento de conexão Wi-Fi 96

Capítulo 3

Classes

3.1 Referência da Estrutura Button

Estrutura que representa um botão físico do sistema.

```
#include <buttons.h>
```

Atributos Públicos

- uint8_t **pin**

  Pino do microcontrolador onde o botão está conectado.
- uint8_t state

  Estado atual do botão após aplicação de debounce.
- uint8_t lastRawReading

  Última leitura bruta do pino (sem debounce).
- unsigned long lastDebounceTime

  Instante da última mudança detectada na leitura bruta.

3.1.1 Descrição detalhada

Estrutura que representa um botão físico do sistema.

Armazena informações necessárias para realizar a leitura do botão com tratamento de debounce por software.

Cada botão mantém o controle do estado atual, da última leitura bruta e do instante da última mudança detectada.

3.1.2 Atributos

3.1.2.1 lastDebounceTime

```
unsigned long Button::lastDebounceTime
```

Instante da última mudança detectada na leitura bruta.

Utilizado para implementar o debounce por software, garantindo que a leitura seja considerada válida somente após um intervalo mínimo de estabilidade.

3.1.2.2 lastRawReading

```
uint8_t Button::lastRawReading
```

Última leitura bruta do pino (sem debounce).

Utilizada para detectar mudanças de estado do botão.

3.1.2.3 state

```
uint8_t Button::state
```

Estado atual do botão após aplicação de debounce.

- LOW → botão pressionado
- HIGH → botão não pressionado

A documentação para essa estrutura foi gerada a partir do seguinte arquivo:

- microcontrolador_central/buttons.h

3.2 Referência da Estrutura ConfigurationData

Estrutura que armazena os parâmetros de configuração do sistema.

```
#include <config_storage.h>
```

Atributos Públicos

- String mqttServer
- String mqttUser
- String mqttPass
- String mqttDeviceId
- int mqttPort
- String wifiPass
- String wifiSSID
- bool valid = false

3.2.1 Descrição detalhada

Estrutura que armazena os parâmetros de configuração do sistema.

Esta estrutura contém todas as informações necessárias para que o dispositivo consiga se conectar à rede WiFi e ao broker MQTT.

Os dados são normalmente carregados do arquivo `/config.json` durante a inicialização do sistema.

O campo `valid` indica se os dados foram carregados corretamente.

3.2.2 Atributos

3.2.2.1 mqttDeviceId

```
String ConfigurationData::mqttDeviceId
```

Identificador único do dispositivo no sistema MQTT

3.2.2.2 mqttPass

```
String ConfigurationData::mqttPass
```

Senha utilizada para autenticação no broker MQTT

3.2.2.3 mqttPort

```
int ConfigurationData::mqttPort
```

Porta de conexão do servidor MQTT (ex.: 1883 ou 8883)

3.2.2.4 mqttServer

```
String ConfigurationData::mqttServer
```

Endereço do servidor MQTT (ex.: "broker.hivemq.com")

3.2.2.5 mqttUser

```
String ConfigurationData::mqttUser
```

Usuário utilizado para autenticação no broker MQTT

3.2.2.6 valid

```
bool ConfigurationData::valid = false
```

Indica se os dados carregados são válidos.

Este campo é utilizado para verificar se a configuração foi carregada corretamente do armazenamento persistente.

3.2.2.7 wifiPass

```
String ConfigurationData::wifiPass
```

Senha da rede WiFi

3.2.2.8 wifiSSID

```
String ConfigurationData::wifiSSID
```

Nome da rede WiFi (SSID)

A documentação para essa estrutura foi gerada a partir do seguinte arquivo:

- microcontrolador_central/config_storage.h

3.3 Referência da Classe List_NursingCall

Classe responsável por gerenciar a lista de chamadas de enfermaria.

```
#include <DoublyLinkedList_NursingCall.h>
```

Membros Públicos

- List_NursingCall ()

  Construtor da lista.
- ~List_NursingCall ()

  Destrutor da lista.
- int getTotal ()

  Retorna o número total de chamadas armazenadas.
- const char * getInfirmaryCurrent ()

  Retorna a enfermaria do nó atualmente selecionado.
- const char * getIdCurrent ()

  Retorna o identificador da chamada atual.
- bool hasNursingCall ()

  Verifica se existem chamadas armazenadas na lista.
- void setDoNotRemoveCurrent (bool value)

  Define o comportamento de proteção do nó atual.
- bool getDoNotRemoveCurrent () const

  Retorna o estado atual da proteção do nó `current`.
- void next ()

  Avança para o próximo nó da lista.
- void prev ()

  Retrocede para o nó anterior da lista.
- bool add (const char *infirmary, const char *id)

  Adiciona uma nova chamada na lista.
- bool removeCurrent ()

  Remove o nó atualmente selecionado.
- bool removalById (const char *id)

  Remove uma chamada pelo seu identificador.
- void clear ()

  Remove todos os elementos da lista.

3.3.1 Descrição detalhada

Classe responsável por gerenciar a lista de chamadas de enfermaria.

Esta classe implementa uma **lista duplamente encadeada** que permite armazenar, navegar e manipular chamadas de enfermaria.

Características principais:

- Navegação sequencial entre chamadas
- Remoção por ID ou pela posição atual
- Controle de limite máximo de chamadas
- Remoção automática da chamada mais antiga quando a lista está cheia
- Proteção opcional contra remoção do nó atualmente selecionado

3.3.2 Construtores e Destrutores

3.3.2.1 List_NursingCall()

```
List_NursingCall::List_NursingCall ()
```

Construtor da lista.

Inicializa a estrutura com a lista vazia.

Inicializa uma lista vazia definindo todos os ponteiros como `nullptr` e o contador de elementos como zero.

3.3.2.2 ∼List_NursingCall()

```
List_NursingCall::~List_NursingCall ()
```

Destrutor da lista.

Libera toda a memória utilizada pelos nós da lista.

Libera toda memória ocupada pelos nós da lista chamando o método `clear()` antes da destruição do objeto.

Isso evita vazamentos de memória.

3.3.3 Documentação das funções

3.3.3.1 add()

```
bool List_NursingCall::add (
            const char * infirmary,
            const char * id)
```

Adiciona uma nova chamada na lista.

Adiciona um novo chamado ao final da lista.

Parâmetros

infirmary	Identificador da enfermaria.
id	Identificador único da chamada.

Retorna

true se a inserção foi realizada com sucesso.

false caso contrário.

Cria um novo nó contendo o número da enfermaria e o ID do dispositivo que gerou o chamado.

Caso a lista atinja o limite máximo (`MAX_CALLS`), chamadas mais antigas são removidas automaticamente seguindo a política FIFO.

Parâmetros

infirmary	Número da enfermaria.
id	Identificador do dispositivo que gerou o chamado.

Retorna

true se o nó foi adicionado com sucesso.

false se ocorrer erro de validação ou alocação.

3.3.3.2 clear()

```
void List_NursingCall::clear ()
```

Remove todos os elementos da lista.

Remove todos os nós da lista.

Libera toda a memória ocupada pelos nós.

Percorre toda a lista liberando a memória de cada nó e reinicializa os ponteiros da lista.

3.3.3.3 getDoNotRemoveCurrent()

```
bool List_NursingCall::getDoNotRemoveCurrent () const
```

Retorna o estado atual da proteção do nó `current`.

Retorna a configuração de proteção do nó atual.

Retorna

true se o nó atual estiver protegido contra remoção automática.

true se o nó atual não pode ser removido automaticamente.

false caso contrário.

3.3.3.4 getIdCurrent()

```
const char * List_NursingCall::getIdCurrent ()
```

Retorna o identificador da chamada atual.

Retorna o ID do dispositivo associado ao nó atual.

Retorna

Ponteiro para string contendo o ID da chamada ou nullptr caso não exista nó atual.

Ponteiro para string contendo o ID ou `nullptr` caso não exista elemento atual.

3.3.3.5 getInfirmaryCurrent()

```
const char * List_NursingCall::getInfirmaryCurrent ()
```

Retorna a enfermaria do nó atualmente selecionado.

Retorna a enfermaria do nó atual.

Retorna

Ponteiro para string com o identificador da enfermaria ou nullptr caso não exista nó atual.

Ponteiro para string contendo o número da enfermaria ou `nullptr` caso não exista elemento atual.

3.3.3.6 getTotal()

```
int List_NursingCall::getTotal ()
```

Retorna o número total de chamadas armazenadas.

Retorna a quantidade total de nós na lista.

Retorna

Quantidade total de nós na lista.

Número de elementos armazenados na lista.

3.3.3.7 hasNursingCall()

```
bool List_NursingCall::hasNursingCall ()
```

Verifica se existem chamadas armazenadas na lista.

Verifica se existem chamados armazenados.

Retorna

true se existir pelo menos uma chamada.
false se a lista estiver vazia.
true se existir pelo menos um chamado.
false se a lista estiver vazia.

3.3.3.8 next()

```
void List_NursingCall::next ()
```

Avança para o próximo nó da lista.

Move o ponteiro atual para o próximo nó.

Atualiza o ponteiro `current` para o próximo elemento.

Caso exista um próximo elemento na lista, o ponteiro `current` é atualizado para ele.

3.3.3.9 prev()

```
void List_NursingCall::prev ()
```

Retrocede para o nó anterior da lista.

Move o ponteiro atual para o nó anterior.

Atualiza o ponteiro `current` para o elemento anterior.

Caso exista um nó anterior na lista, o ponteiro `current` é atualizado para ele.

3.3.3.10 removalById()

```
bool List_NursingCall::removalById (
            const char * id)
```

Remove uma chamada pelo seu identificador.

Remove um chamado utilizando o ID do dispositivo.

Parâmetros

id	Identificador da chamada.

Retorna

true se a chamada foi encontrada e removida.

false caso contrário.

Parâmetros

id	Identificador do dispositivo.

Retorna

true se o elemento foi encontrado e removido.

false se nenhum elemento com esse ID existir.

3.3.3.11 removeCurrent()

```
bool List_NursingCall::removeCurrent ()
```

Remove o nó atualmente selecionado.

Remove o nó atual da lista.

Retorna

true se a remoção foi realizada com sucesso.

false caso não exista nó atual.

true se a remoção ocorrer com sucesso.

3.3.3.12 setDoNotRemoveCurrent()

```
void List_NursingCall::setDoNotRemoveCurrent (
            bool value)
```

Define o comportamento de proteção do nó atual.

Define se o nó atual pode ser removido automaticamente.

Parâmetros

value	• true → impede remoção automática do nó `current` • false → permite remoção automática

Este parâmetro controla a política de remoção automática quando a lista atinge seu limite máximo de elementos.

Parâmetros

value	Valor booleano indicando permissão de remoção.

A documentação para essa classe foi gerada a partir dos seguintes arquivos:

- microcontrolador_central/DoublyLinkedList_NursingCall.h
- microcontrolador_central/DoublyLinkedList_NursingCall.cpp

3.4 Referência da Estrutura NursingCall

Estrutura que representa um nó da lista de chamadas.

```
#include <DoublyLinkedList_NursingCall.h>
```

Atributos Públicos

- char infirmary [32]
- char id [32]
- NursingCall * next
- NursingCall * prev

3.4.1 Descrição detalhada

Estrutura que representa um nó da lista de chamadas.

Cada nó armazena informações referentes a uma chamada de enfermaria e mantém ponteiros para os nós anterior e posterior, formando uma lista duplamente encadeada.

3.4.2 Atributos

3.4.2.1 id

```
char NursingCall::id[32]
```

Identificador único da chamada

3.4.2.2 infirmary

```
char NursingCall::infirmary[32]
```

Identificador textual da enfermaria

3.4.2.3 next

```
NursingCall* NursingCall::next
```

Ponteiro para o próximo nó da lista

3.4.2.4 prev

```
NursingCall* NursingCall::prev
```

Ponteiro para o nó anterior da lista

A documentação para essa estrutura foi gerada a partir do seguinte arquivo:

- microcontrolador_central/DoublyLinkedList_NursingCall.h

Capítulo 4

Arquivos

4.1 Referência do Arquivo microcontrolador_central/buttons.cpp

Implementação do gerenciamento de botões do sistema.

```
#include "buttons.h"
```

Funções

- void IRAM_ATTR longPressButtonISR ()

  Rotina de interrupção para detecção de pressionamento longo.
- void initButtons ()

  Inicializa os botões do sistema.
- bool checkButton (Button &btn)

  Verifica se um botão foi pressionado.
- void enableButtonInterruptChange ()

  Habilita a interrupção para detecção de long press.
- bool isConfigurationMode ()

  Verifica se o sistema está em modo de configuração.

Variáveis

- constexpr unsigned long DEBOUNCE_DELAY = 50

  Tempo mínimo para debounce dos botões.
- unsigned long buttonPressStartTime = 0

  Armazena o instante em que o botão foi pressionado.
- volatile bool isButtonPressed = false

  Indica se o botão está atualmente pressionado.
- volatile bool isButtonLongPressActivate = false

  Flag que indica se o modo de configuração foi ativado.
- Button button_next = { 14, HIGH, HIGH, 0 }

  Instância do botão "Next".
- Button button_prev = { 12, HIGH, HIGH, 0 }

  Instância do botão "Previous".
- Button button_delete = { 13, HIGH, HIGH, 0 }

  Instância do botão "Delete".

4.1.1 Descrição detalhada

Implementação do gerenciamento de botões do sistema.

Este módulo implementa a leitura e o controle de múltiplos botões físicos utilizados para interação com o usuário.

Os botões disponíveis no sistema são:

- Botão **Next** → navegação para próximo item
- Botão **Prev** → navegação para item anterior
- Botão **Delete** → remoção de item

Funcionalidades implementadas:

- Leitura de botões com debounce por software
- Detecção de pressionamento (evento de clique)
- Detecção de pressionamento longo (long press)
- Uso de interrupções para ativação de modo de configuração

O debounce é realizado por software utilizando um intervalo mínimo entre leituras válidas, evitando múltiplas detecções causadas pelo ruído mecânico dos botões.

O botão **Prev** também é utilizado para detectar um pressionamento longo (15 segundos), que alterna o modo de configuração do sistema.

Observação

Este módulo depende da estrutura `Button` definida em `buttons.h`.

Aviso

Variáveis utilizadas em rotinas de interrupção são declaradas como `volatile` para garantir comportamento correto em ambientes concorrentes.

Data

2026

4.1.2 Funções

4.1.2.1 checkButton()

```
bool checkButton (
            Button & btn)
```

Verifica se um botão foi pressionado.

Parâmetros

btn	Referência para a estrutura do botão.

Retorna

true Se um pressionamento válido foi detectado.

false Caso contrário.

Implementa debounce por software verificando se o estado do botão permaneceu estável por um período mínimo definido em `DEBOUNCE_DELAY`.

A função retorna `true` apenas no momento em que o botão é pressionado (transição HIGH → LOW).

4.1.2.2 enableButtonInterruptChange()

```
void enableButtonInterruptChange ()
```

Habilita a interrupção para detecção de long press.

Habilita interrupção para detecção de pressionamento longo.

Associa o pino do botão `button_prev` à rotina de interrupção responsável por detectar pressionamentos longos.

4.1.2.3 initButtons()

```
void initButtons ()
```

Inicializa os botões do sistema.

Configura os pinos dos botões como entrada com resistor pull-up interno ativado.

Com essa configuração:

- HIGH → botão não pressionado
- LOW → botão pressionado

4.1.2.4 isConfigurationMode()

```
bool isConfigurationMode ()
```

Verifica se o sistema está em modo de configuração.

Retorna

true Se o modo de configuração está ativo.

false Caso contrário.

O modo de configuração é ativado ou desativado através de um pressionamento longo do botão `button_prev`.

4.1.2.5 longPressButtonISR()

```
void IRAM_ATTR longPressButtonISR ()
```

Rotina de interrupção para detecção de pressionamento longo.

Esta função é executada automaticamente quando ocorre uma mudança de estado (CHANGE) no pino do botão `button_prev`.

Funcionamento:

- Quando o botão é pressionado (LOW), registra o tempo atual.
- Quando o botão é liberado (HIGH), calcula o tempo pressionado.
- Se o tempo for maior ou igual a **15 segundos**, o modo de configuração do sistema é alternado.

Observação

Esta função é executada em contexto de interrupção e deve permanecer o mais curta possível.

4.1.3 Variáveis

4.1.3.1 button_delete

```
Button button_delete = { 13, HIGH, HIGH, 0 }
```

Instância do botão "Delete".

Utilizado para remover registros ou itens selecionados.

4.1.3.2 button_next

```
Button button_next = { 14, HIGH, HIGH, 0 }
```

Instância do botão "Next".

Utilizado para navegar para o próximo item ou registro.

4.1.3.3 button_prev

```
Button button_prev = { 12, HIGH, HIGH, 0 }
```

Instância do botão "Previous".

Utilizado para navegar para o item anterior. Também é responsável por detectar o pressionamento longo que ativa o modo de configuração do sistema.

4.1.3.4 buttonPressStartTime

```
unsigned long buttonPressStartTime = 0
```

Armazena o instante em que o botão foi pressionado.

Utilizado para calcular a duração do pressionamento do botão e detectar eventos de long press.

4.1.3.5 DEBOUNCE_DELAY

```
unsigned long DEBOUNCE_DELAY = 50  [constexpr]
```

Tempo mínimo para debounce dos botões.

Define o intervalo mínimo (em milissegundos) que o sinal do botão deve permanecer estável antes de ser considerado uma leitura válida.

Isso evita múltiplas leituras causadas pelo ruído mecânico do contato físico do botão.

4.1.3.6 isButtonLongPressActivate

```
volatile bool isButtonLongPressActivate = false
```

Flag que indica se o modo de configuração foi ativado.

Esta variável alterna seu estado quando ocorre um pressionamento longo do botão configurado.

4.1.3.7 isButtonPressed

```
volatile bool isButtonPressed = false
```

Indica se o botão está atualmente pressionado.

Esta variável é utilizada durante a rotina de interrupção para detectar a transição entre pressionado e liberado.

4.2 Referência do Arquivo microcontrolador_central/buttons.h

Interface do módulo de gerenciamento de botões físicos.

```
#include "log.h"
#include <Arduino.h>
```

Componentes

- struct Button

  Estrutura que representa um botão físico do sistema.

Funções

- void initButtons ()

  Inicializa os botões do sistema.
- bool checkButton (Button &btn)

  Verifica se um botão foi pressionado.
- void enableButtonInterruptChange ()

  Habilita interrupção para detecção de pressionamento longo.
- bool isConfigurationMode ()

  Verifica se o sistema está em modo de configuração.

Variáveis

- Button button_next

  Instância do botão "Next".
- Button button_prev

  Instância do botão "Previous".
- Button button_delete

  Instância do botão "Delete".

4.2.1 Descrição detalhada

Interface do módulo de gerenciamento de botões físicos.

Este módulo define a estrutura e as funções responsáveis pela leitura e gerenciamento de botões físicos no sistema.

Os botões são utilizados para interação com o usuário, permitindo navegação entre registros e execução de ações como exclusão ou entrada no modo de configuração.

Funcionalidades fornecidas:

- Estrutura de dados para controle de botões
- Leitura com debounce por software
- Inicialização dos pinos de entrada
- Detecção de eventos de pressionamento
- Detecção de pressionamento longo via interrupção

O módulo trabalha com três botões principais:

- **button_next** → navegação para o próximo item
- **button_prev** → navegação para o item anterior
- **button_delete** → exclusão de registros

O botão `button_prev` também é utilizado para ativar o modo de configuração do sistema através de um pressionamento longo.

Observação

A implementação das funções está no arquivo `buttons.cpp`.

Data

2026

4.2.2 Funções

4.2.2.1 checkButton()

```
bool checkButton (
            Button & btn)  [extern]
```

Verifica se um botão foi pressionado.

Parâmetros

btn	Referência para a estrutura do botão.

Retorna

true Se um pressionamento válido foi detectado.

false Caso contrário.

Implementa debounce por software verificando se o estado do botão permaneceu estável por um período mínimo antes de confirmar o evento de pressionamento.

A função retorna `true` apenas quando detecta uma transição válida para o estado pressionado.

Parâmetros

btn	Referência para a estrutura do botão.

Retorna

true Se um pressionamento válido foi detectado.

false Caso contrário.

Implementa debounce por software verificando se o estado do botão permaneceu estável por um período mínimo definido em `DEBOUNCE_DELAY`.

A função retorna `true` apenas no momento em que o botão é pressionado (transição HIGH $\rightarrow$ LOW).

4.2.2.2 enableButtonInterruptChange()

```
void enableButtonInterruptChange ()
```

Habilita interrupção para detecção de pressionamento longo.

Associa o botão responsável pelo modo de configuração à rotina de interrupção que monitora mudanças de estado do pino e calcula a duração do pressionamento.

Habilita interrupção para detecção de pressionamento longo.

Associa o pino do botão `button_prev` à rotina de interrupção responsável por detectar pressionamentos longos.

4.2.2.3 initButtons()

```
void initButtons () [extern]
```

Inicializa os botões do sistema.

Configura os pinos definidos nas estruturas `Button` como entradas com resistor pull-up interno ativado.

Observação

Deve ser chamada durante o `setup()` do sistema.

Configura os pinos dos botões como entrada com resistor pull-up interno ativado.

Com essa configuração:

- HIGH → botão não pressionado
- LOW → botão pressionado

4.2.2.4 isConfigurationMode()

```
bool isConfigurationMode ()
```

Verifica se o sistema está em modo de configuração.

Retorna

true Se o modo de configuração estiver ativo.

false Caso contrário.

O modo de configuração é ativado através de um pressionamento longo do botão `button_prev`.

Retorna

true Se o modo de configuração está ativo.

false Caso contrário.

O modo de configuração é ativado ou desativado através de um pressionamento longo do botão `button_prev`.

4.2.3 Variáveis

4.2.3.1 button_delete

```
Button button_delete [extern]
```

Instância do botão "Delete".

Utilizado para remover registros ou itens selecionados.

4.2.3.2 button_next

```
Button button_next [extern]
```

Instância do botão "Next".

Utilizado para navegar para o próximo item ou registro.

4.2.3.3 button_prev

```
Button button_prev [extern]
```

Instância do botão "Previous".

Utilizado para navegar para o item anterior. Também é responsável por detectar pressionamento longo para ativação do modo de configuração.

Utilizado para navegar para o item anterior. Também é responsável por detectar o pressionamento longo que ativa o modo de configuração do sistema.

4.3 buttons.h

Ir para a documentação desse arquivo.

```
00001
00036
00037 #ifndef BUTTONS_H
00038 #define BUTTONS_H
00039
00040 #include "log.h"
00041 #include <Arduino.h>
00042
00054 struct Button {
00055
00059   uint8_t pin;
00060
00067   uint8_t state;
00068
00074   uint8_t lastRawReading;
00075
00083   unsigned long lastDebounceTime;
00084 };
00085
00086
00092 extern Button button_next;
00093
00101 extern Button button_prev;
00102
00108 extern Button button_delete;
00109
00118 extern void initButtons();
00119
00120
00137 extern bool checkButton(Button &btn);
00138
00139
00147 void enableButtonInterruptChange();
00148
00149
00160 bool isConfigurationMode();
00161
00162 #endif
```

4.4 Referência do Arquivo microcontrolador_central/buzzer.cpp

Implementação do módulo de controle do buzzer.

```
#include "buzzer.h"
#include "log.h"
#include "config.h"
```

Funções

- void buzzerInit ()

  Inicializa o buzzer.
- void toggleBuzzer ()

  Alterna o estado do buzzer para gerar um alerta sonoro.
- bool doesHaveNotificationBuzzer ()

  Verifica se existe uma notificação sonora ativa.
- void enableSoundAlert ()

  Ativa um novo alerta sonoro no sistema.

Variáveis

- bool isBuzzerNotificationEnabled = false

  Indica se existe uma notificação sonora ativa.
- uint8_t numberAlertIssued = 0

  Contador de alertas sonoros já emitidos.
- unsigned long lastBuzzerPlayTime = 0

  Armazena o instante da última alternância do buzzer.

4.4.1 Descrição detalhada

Implementação do módulo de controle do buzzer.

Este módulo é responsável por controlar o buzzer utilizado para emitir alertas sonoros no sistema.

As funcionalidades incluem:

- Inicialização do hardware do buzzer
- Geração de alertas sonoros intermitentes
- Controle do número máximo de alertas emitidos
- Gerenciamento do estado de notificação sonora

O buzzer alterna seu estado (ligado/desligado) em intervalos definidos pela constante `BUZZER_INTERVAL_MS`, gerando um padrão de som intermitente.

O número máximo de ciclos de alerta é controlado pela constante `NUMBER_SOUND_ALERTS`.

O controle de tempo é realizado utilizando a função `millis()`, evitando o uso de delays bloqueantes e permitindo que o sistema continue executando outras tarefas.

Observação

As definições de pinos e constantes são obtidas de `config.h`.

Data

2026

4.4.2 Funções

4.4.2.1 buzzerInit()

```
void buzzerInit ()
```

Inicializa o buzzer.

Inicializa o hardware do buzzer.

Configura o pino do buzzer como saída digital e garante que o dispositivo inicie desligado.

Observação

Deve ser chamada durante o `setup()` do sistema.

4.4.2.2 doesHaveNotificationBuzzer()

```
bool doesHaveNotificationBuzzer ()
```

Verifica se existe uma notificação sonora ativa.

Retorna

true Se o sistema possui um alerta sonoro ativo.

false Caso contrário.

4.4.2.3 enableSoundAlert()

```
void enableSoundAlert ()
```

Ativa um novo alerta sonoro no sistema.

Ativa um novo alerta sonoro.

Habilita a emissão de alertas pelo buzzer e reinicia o contador de ciclos de som.

Após chamada desta função, o buzzer começará a emitir alertas quando `toggleBuzzer()` for executado.

4.4.2.4 toggleBuzzer()

```
void toggleBuzzer ()
```

Alterna o estado do buzzer para gerar um alerta sonoro.

Controla a alternância do buzzer durante um alerta.

Esta função controla o funcionamento do buzzer através de alternâncias periódicas entre os estados HIGH e LOW.

O intervalo entre alternâncias é definido pela constante `BUZZER_INTERVAL_MS`.

Funcionamento:

1. Verifica se o intervalo de tempo mínimo foi atingido.
2. Alterna o estado do buzzer.
3. Incrementa o contador de alertas quando o buzzer é ativado.
4. Desativa automaticamente o alerta quando o número máximo de ciclos definido em `NUMBER_SOUND_ALERTS` é atingido.

Observação

Deve ser chamada periodicamente no `loop()` principal.

4.4.3 Variáveis

4.4.3.1 isBuzzerNotificationEnabled

```
bool isBuzzerNotificationEnabled = false
```

Indica se existe uma notificação sonora ativa.

Quando `true`, o buzzer deve emitir alertas sonoros até atingir o número máximo definido de ciclos.

4.4.3.2 lastBuzzerPlayTime

```
unsigned long lastBuzzerPlayTime = 0
```

Armazena o instante da última alternância do buzzer.

Utilizada para controlar o intervalo entre mudanças de estado do buzzer, garantindo que ele seja acionado em uma frequência adequada.

O controle é baseado na função `millis()`, permitindo operação não bloqueante do sistema.

4.4.3.3 numberAlertIssued

```
uint8_t numberAlertIssued = 0
```

Contador de alertas sonoros já emitidos.

Cada vez que o buzzer é ativado (estado HIGH), o contador é incrementado. Quando o número máximo definido em `NUMBER_SOUND_ALERTS` é atingido, o alerta sonoro é automaticamente desativado.

4.5 Referência do Arquivo microcontrolador_central/buzzer.h

Interface do módulo de controle do buzzer.

```
#include <Arduino.h>
```

Funções

- void buzzerInit ()

  Inicializa o hardware do buzzer.
- void toggleBuzzer ()

  Controla a alternância do buzzer durante um alerta.
- void enableSoundAlert ()

  Ativa um novo alerta sonoro.
- bool doesHaveNotificationBuzzer ()

  Verifica se existe uma notificação sonora ativa.

Variáveis

- constexpr unsigned long BUZZER_INTERVAL_MS = 1000

  Intervalo de tempo entre alternâncias do buzzer.

4.5.1 Descrição detalhada

Interface do módulo de controle do buzzer.

Este módulo define a interface responsável pelo controle do buzzer utilizado no sistema para emissão de alertas sonoros.

O buzzer é acionado quando o sistema detecta uma condição que exige notificação ao usuário, como alarmes ou eventos importantes.

Funcionalidades disponíveis:

- Inicialização do hardware do buzzer
- Ativação de alertas sonoros
- Alternância periódica do estado do buzzer
- Verificação do estado atual de notificação sonora

O controle de tempo é baseado na função `millis()`, permitindo operação não bloqueante dentro do loop principal do firmware.

Observação

As definições de pinos do buzzer são obtidas em `config.h`.

Data

2026

4.5.2 Funções

4.5.2.1 buzzerInit()

```
void buzzerInit ()
```

Inicializa o hardware do buzzer.

Configura o pino do buzzer como saída digital e garante que o dispositivo inicie desligado.

Esta função deve ser chamada no `setup()` antes de qualquer tentativa de uso do buzzer.

Inicializa o hardware do buzzer.

Configura o pino do buzzer como saída digital e garante que o dispositivo inicie desligado.

Observação

Deve ser chamada durante o `setup()` do sistema.

4.5.2.2 doesHaveNotificationBuzzer()

```
bool doesHaveNotificationBuzzer ()
```

Verifica se existe uma notificação sonora ativa.

Retorna

true Se o buzzer possui um alerta ativo.

false Caso contrário.

true Se o sistema possui um alerta sonoro ativo.

false Caso contrário.

4.5.2.3 enableSoundAlert()

```
void enableSoundAlert ()
```

Ativa um novo alerta sonoro.

Habilita a emissão de alertas pelo buzzer. Após chamada desta função, o buzzer começará a alternar seu estado quando `toggleBuzzer()` for executada.

Ativa um novo alerta sonoro.

Habilita a emissão de alertas pelo buzzer e reinicia o contador de ciclos de som.

Após chamada desta função, o buzzer começará a emitir alertas quando `toggleBuzzer()` for executado.

4.5.2.4 toggleBuzzer()

```
void toggleBuzzer ()
```

Controla a alternância do buzzer durante um alerta.

Alterna periodicamente o estado do buzzer para gerar um padrão sonoro intermitente.

A alternância ocorre apenas quando o intervalo definido em `BUZZER_INTERVAL_MS` é atingido.

Observação

Deve ser chamada periodicamente dentro do `loop()` principal.

Controla a alternância do buzzer durante um alerta.

Esta função controla o funcionamento do buzzer através de alternâncias periódicas entre os estados HIGH e LOW.

O intervalo entre alternâncias é definido pela constante `BUZZER_INTERVAL_MS`.

Funcionamento:

1. Verifica se o intervalo de tempo mínimo foi atingido.
2. Alterna o estado do buzzer.
3. Incrementa o contador de alertas quando o buzzer é ativado.
4. Desativa automaticamente o alerta quando o número máximo de ciclos definido em `NUMBER_SOUND_ALERTS` é atingido.

Observação

Deve ser chamada periodicamente no `loop()` principal.

4.5.3 Variáveis

4.5.3.1 BUZZER_INTERVAL_MS

```
unsigned long BUZZER_INTERVAL_MS = 1000  [constexpr]
```

Intervalo de tempo entre alternâncias do buzzer.

Define o tempo, em milissegundos, entre cada mudança de estado do buzzer (HIGH → LOW ou LOW → HIGH).

Esse valor controla a frequência do alerta sonoro gerado.

Valor atual: **1000 ms (1 segundo)**.

4.6 buzzer.h

Ir para a documentação desse arquivo.

```
00001
00028
00029 #ifndef BUZZER_H
00030 #define BUZZER_H
00031
00032 #include <Arduino.h>
00033
00034
00035
00047 constexpr unsigned long BUZZER_INTERVAL_MS = 1000;
00048
00049
00060 void buzzerInit();
00061
00062
00075 void toggleBuzzer();
00076
00077
00086 void enableSoundAlert();
00087
00088
00095 bool doesHaveNotificationBuzzer();
00096
00097
00098 #endif
```

4.7 Referência do Arquivo microcontrolador_central/config.h

Arquivo central de configuração do firmware.

```
#include "log.h"
```

Variáveis

- constexpr LogLevel LOG_MODE = LOG_DEBUG

  Define o nível de verbosidade do sistema de log.
- constexpr uint8_t PIN_BUZZER = 15

  Pino digital utilizado para controle do buzzer.
- constexpr uint8_t PIN_LED = 2

  Pino digital utilizado para controle do LED indicador.
- constexpr uint8_t NUMBER_SOUND_ALERTS = 5

  Número máximo de ciclos de alerta sonoro emitidos.
- constexpr const char ∗ **MQTT_PUBLICATION_TOPIC** = "dispositivos/enfermaria"

  Tópico MQTT utilizado para publicação de eventos do dispositivo.
- constexpr const char ∗ **MQTT_SUBSCRIPTION_TOPIC** = "dispositivos/posto_enfermaria"

  Tópico MQTT utilizado para recebimento de comandos do sistema central.
- constexpr const char ∗ **MQTT_PUB_CONFIRMATION_TOPIC** = "dispositivo/confirmacao"

  Tópico MQTT utilizado para envio de confirmação de mensagens.
- constexpr const char ∗ **MQTT_SUB_CONFIRMATION_TOPIC** = "dispositivo/confirmacao/posto_enfermaria"

  Tópico MQTT utilizado para recebimento de confirmação do servidor.
- constexpr int MQTT_PORT = 1883

  Porta padrão utilizada para comunicação com o broker MQTT.
- constexpr const char ∗ SSID_ACCESS_POINT = "xxxxxxxxxxxxx"

  Nome da rede WiFi criada pelo dispositivo (Access Point).
- constexpr const char ∗ PASSWORD_ACCESS_POINT = "xxxxxxxxxxxx"

  Senha da rede WiFi criada pelo dispositivo.

4.7.1 Descrição detalhada

Arquivo central de configuração do firmware.

Este arquivo define constantes utilizadas em todo o sistema, incluindo:

- Configuração do sistema de log
- Definição de pinos de hardware
- Parâmetros de alerta sonoro
- Tópicos de comunicação MQTT
- Configurações da rede WiFi de acesso do dispositivo

Essas definições são utilizadas por diferentes módulos do firmware e centralizam os parâmetros que podem precisar ser alterados durante o desenvolvimento ou implantação do sistema.

Observação

Alterações neste arquivo podem impactar diversos módulos do sistema.

Data

2026

4.7.2 Variáveis

4.7.2.1 LOG_MODE

```
LogLevel LOG_MODE = LOG_DEBUG  [constexpr]
```

Define o nível de verbosidade do sistema de log.

Controla quais mensagens serão exibidas no monitor serial. Isso é útil para depuração e monitoramento do sistema.

Níveis disponíveis (em ordem crescente de verbosidade):

- LOG_NONE : desativa logs
- LOG_ERROR : apenas erros críticos
- LOG_WARN : avisos e erros
- LOG_INFO : informações gerais do sistema
- LOG_DEBUG : mensagens detalhadas para depuração

4.7.2.2 MQTT_PORT

```
int MQTT_PORT = 1883  [constexpr]
```

Porta padrão utilizada para comunicação com o broker MQTT.

Valor padrão do protocolo MQTT: **1883**.

4.7.2.3 NUMBER_SOUND_ALERTS

```
uint8_t NUMBER_SOUND_ALERTS = 5  [constexpr]
```

Número máximo de ciclos de alerta sonoro emitidos.

Define quantas vezes o buzzer será ativado durante um evento de alerta antes de ser automaticamente desativado.

4.7.2.4 PASSWORD_ACCESS_POINT

```
const char* PASSWORD_ACCESS_POINT = "xxxxxxxxxxxx"  [constexpr]
```

Senha da rede WiFi criada pelo dispositivo.

Utilizada para proteger o acesso ao ponto de acesso gerado pelo microcontrolador.

4.7.2.5 PIN_BUZZER

```
uint8_t PIN_BUZZER = 15  [constexpr]
```

Pino digital utilizado para controle do buzzer.

Este pino é configurado como saída e utilizado para gerar alertas sonoros no sistema.

4.7.2.6 PIN_LED

```
uint8_t PIN_LED = 2  [constexpr]
```

Pino digital utilizado para controle do LED indicador.

O LED pode ser utilizado para indicar estados do sistema, como conexão WiFi, atividade do dispositivo ou condições de alerta.

4.7.2.7 SSID_ACCESS_POINT

```
const char* SSID_ACCESS_POINT = "xxxxxxxxxxxxx"  [constexpr]
```

Nome da rede WiFi criada pelo dispositivo (Access Point).

Esta rede é utilizada para acesso às configurações internas do dispositivo, normalmente durante a etapa de configuração inicial ou manutenção.

4.8 config.h

Ir para a documentação desse arquivo.

```
00001
00025
00026 #ifndef CONFIG_HELPER_H
00027 #define CONFIG_HELPER_H
00028
00029 #include "log.h"
00030
00031
00032
00048 constexpr LogLevel LOG_MODE = LOG_DEBUG;
00049
00050
00051
00052 // ------------------------------------------------------------------------------
00053 // Definição de pinos de hardware
00054 // ------------------------------------------------------------------------------
00055
00063 constexpr uint8_t PIN_BUZZER = 15;
00064
00065
00074 constexpr uint8_t PIN_LED = 2;
00075
00076
00077
00078 // ------------------------------------------------------------------------------
00079 // Configuração de alertas sonoros
00080 // ------------------------------------------------------------------------------
00081
00090 constexpr uint8_t NUMBER_SOUND_ALERTS = 5;
00091
00092
00093
00094 // ------------------------------------------------------------------------------
00095 // Configuração de tópicos MQTT
```

```
// ---------------------------------------------------------------------------

constexpr const char* MQTT_PUBLICATION_TOPIC = "dispositivos/enfermaria";


constexpr const char* MQTT_SUBSCRIPTION_TOPIC = "dispositivos/posto_enfermaria";


constexpr const char* MQTT_PUB_CONFIRMATION_TOPIC = "dispositivo/confirmacao";


constexpr const char* MQTT_SUB_CONFIRMATION_TOPIC = "dispositivo/confirmacao/posto_enfermaria";


constexpr int MQTT_PORT = 1883;



// ---------------------------------------------------------------------------
// Configuração da rede WiFi de acesso do dispositivo
// ---------------------------------------------------------------------------

constexpr const char* SSID_ACCESS_POINT = "xxxxxxxxxxxxx";


constexpr const char* PASSWORD_ACCESS_POINT = "xxxxxxxxxxxx";


#endif
```

4.9 Referência do Arquivo microcontrolador_central/config_storage.cpp

Implementação do módulo de armazenamento de configurações do sistema.

```
#include "config_storage.h"
#include <ArduinoJson.h>
#include <LittleFS.h>
#include "log.h"
```

Funções

- void initConfigStorage ()

  Inicializa o subsistema de armazenamento de configuração.
- bool hasConfigData ()

  Verifica se existe um arquivo de configuração salvo.
- ConfigurationData loadConfig ()

  Carrega os dados de configuração do arquivo JSON.
- bool saveConfigutionData (const ConfigurationData &config)

  Salva os dados de configuração no arquivo JSON.

Variáveis

- const char ∗ CONFIG_PATH = "/config.json"

  Caminho do arquivo de configuração no sistema de arquivos.
- ConfigurationData cfg

  Estrutura global contendo a configuração atual do sistema.

4.9.1 Descrição detalhada

Implementação do módulo de armazenamento de configurações do sistema.

Este módulo é responsável por gerenciar o armazenamento persistente das configurações do dispositivo utilizando o sistema de arquivos interno LittleFS.

As configurações são armazenadas no arquivo `/config.json` em formato JSON e incluem parâmetros necessários para o funcionamento do sistema, como:

- Credenciais da rede WiFi
- Configurações do servidor MQTT
- Identificação do dispositivo

O módulo fornece funcionalidades para:

- Inicializar o subsistema de armazenamento
- Verificar se existe configuração salva
- Carregar configurações do arquivo
- Salvar novas configurações

A biblioteca ArduinoJson é utilizada para realizar a serialização e desserialização dos dados em formato JSON.

O uso do LittleFS permite que as configurações permaneçam armazenadas mesmo após reinicializações do dispositivo.

Observação

Este módulo depende do sistema de arquivos LittleFS.

Data

2026

4.9.2 Funções

4.9.2.1 hasConfigData()

```
bool hasConfigData ()
```

Verifica se existe um arquivo de configuração salvo.

Essa função verifica se o arquivo `/config.json` está presente no sistema de arquivos interno.

Retorna

true Se o arquivo de configuração existir.

false Caso o arquivo não exista.

4.9.2.2 initConfigStorage()

```
void initConfigStorage ()
```

Inicializa o subsistema de armazenamento de configuração.

Inicializa o módulo de armazenamento de configuração.

Essa função inicializa o sistema de arquivos LittleFS, responsável por armazenar o arquivo de configuração do dispositivo.

Caso o sistema de arquivos não consiga ser montado, uma mensagem de erro é registrada no sistema de log.

Observação

Deve ser chamada durante a inicialização do sistema (normalmente no `setup()`).

4.9.2.3 loadConfig()

```
ConfigurationData loadConfig ()
```

Carrega os dados de configuração do arquivo JSON.

Carrega os dados de configuração armazenados.

Abre o arquivo `/config.json`, lê seu conteúdo e converte os valores armazenados para a estrutura `ConfigurationData`.

O processo consiste em:

1. Abrir o arquivo de configuração.
2. Desserializar o conteúdo JSON.
3. Mapear os campos JSON para a estrutura de configuração.
4. Retornar a estrutura preenchida.

Caso ocorra erro em qualquer etapa, uma estrutura inválida é retornada (`valid = false`).

Retorna

Estrutura `ConfigurationData` contendo os dados carregados.

Documento JSON utilizado para leitura da configuração.

O tamanho foi definido como 768 bytes para acomodar todos os campos esperados no arquivo de configuração.

Indica que os dados foram carregados corretamente.

4.9.2.4 saveConfigutionData()

```
bool saveConfigutionData (
            const ConfigurationData & config)
```

Salva os dados de configuração no arquivo JSON.

Salva os dados de configuração no armazenamento persistente.

Essa função cria ou sobrescreve o arquivo `/config.json` no sistema de arquivos LittleFS, armazenando os dados contidos na estrutura `ConfigurationData`.

O processo consiste em:

1. Converter a estrutura de configuração para JSON.
2. Abrir o arquivo em modo de escrita.
3. Serializar o JSON no arquivo.
4. Recarregar a configuração salva na variável global `cfg`.

O uso de `serializeJsonPretty()` facilita a leitura manual do arquivo durante depuração.

Parâmetros

config	Estrutura contendo os dados de configuração.

Retorna

true Se os dados foram salvos corretamente.
false Caso ocorra erro ao abrir ou escrever o arquivo.

Abre o arquivo de configuração em modo escrita.

Caso o arquivo já exista, ele será sobrescrito.

Recarrega a configuração para manter a variável global `cfg` sincronizada com o conteúdo salvo.

4.9.3 Variáveis

4.9.3.1 cfg

```
ConfigurationData cfg
```

Estrutura global contendo a configuração atual do sistema.

Essa variável mantém os dados de configuração carregados do arquivo `/config.json`.

Ela é atualizada quando:

- O sistema carrega a configuração do armazenamento
- Uma nova configuração é salva no dispositivo

A estrutura `ConfigurationData` contém parâmetros como:

- Credenciais WiFi
- Configurações do broker MQTT
- Identificação do dispositivo

4.9.3.2 CONFIG_PATH

```
const char* CONFIG_PATH = "/config.json"
```

Caminho do arquivo de configuração no sistema de arquivos.

Define o local onde os dados de configuração do dispositivo são armazenados no sistema de arquivos interno (LittleFS).

O arquivo utiliza o formato JSON para armazenar parâmetros como credenciais WiFi e configurações do servidor MQTT.

4.10 Referência do Arquivo microcontrolador_central/config_storage.h

Interface do módulo de armazenamento persistente de configuração.

```
#include <Arduino.h>
```

Componentes

- struct ConfigurationData

  Estrutura que armazena os parâmetros de configuração do sistema.

Funções

- void initConfigStorage ()

  Inicializa o módulo de armazenamento de configuração.
- ConfigurationData loadConfig ()

  Carrega os dados de configuração armazenados.
- bool saveConfigutionData (const ConfigurationData &config)

  Salva os dados de configuração no armazenamento persistente.
- bool hasConfigData ()

  Verifica se existe um arquivo de configuração salvo.

Variáveis

- ConfigurationData cfg

  Estrutura global contendo a configuração atual do sistema.

4.10.1 Descrição detalhada

Interface do módulo de armazenamento persistente de configuração.

Este módulo define a estrutura de dados utilizada para armazenar as configurações do sistema e fornece funções responsáveis por salvar e carregar essas informações utilizando o sistema de arquivos interno LittleFS.

As configurações são armazenadas em formato JSON no arquivo `/config.json`, permitindo persistência mesmo após reinicializações do dispositivo.

Entre os dados armazenados estão:

- Credenciais da rede WiFi
- Parâmetros de conexão com o servidor MQTT
- Identificação do dispositivo no sistema

O módulo fornece funções para:

- Inicializar o subsistema de armazenamento
- Verificar se existe configuração salva
- Carregar configurações do armazenamento
- Salvar novas configurações

Observação

Este módulo depende do sistema de arquivos LittleFS.

Data

2026

4.10.2 Funções

4.10.2.1 hasConfigData()

```
bool hasConfigData ()
```

Verifica se existe um arquivo de configuração salvo.

Confere se o arquivo `/config.json` está presente no sistema de arquivos LittleFS.

Retorna

true Se o arquivo existir.
false Caso contrário.

Essa função verifica se o arquivo `/config.json` está presente no sistema de arquivos interno.

Retorna

true Se o arquivo de configuração existir.
false Caso o arquivo não exista.

4.10.2.2 initConfigStorage()

```
void initConfigStorage ()
```

Inicializa o módulo de armazenamento de configuração.

Prepara o subsistema responsável por gerenciar as configurações persistentes do dispositivo.

Esta função normalmente realiza a inicialização do sistema de arquivos LittleFS, permitindo leitura e escrita no arquivo de configuração.

Observação

Deve ser chamada durante a inicialização do sistema (geralmente dentro do `setup()`).

Inicializa o módulo de armazenamento de configuração.

Essa função inicializa o sistema de arquivos LittleFS, responsável por armazenar o arquivo de configuração do dispositivo.

Caso o sistema de arquivos não consiga ser montado, uma mensagem de erro é registrada no sistema de log.

Observação

Deve ser chamada durante a inicialização do sistema (normalmente no `setup()`).

4.10.2.3 loadConfig()

```
ConfigurationData loadConfig ()
```

Carrega os dados de configuração armazenados.

Lê o arquivo `/config.json` presente no sistema de arquivos LittleFS e converte seu conteúdo para uma estrutura `ConfigurationData`.

Caso o arquivo não exista ou ocorra erro durante a leitura, será retornada uma estrutura com `valid = false`.

Retorna

Estrutura `ConfigurationData` contendo os dados carregados.

Carrega os dados de configuração armazenados.

Abre o arquivo `/config.json`, lê seu conteúdo e converte os valores armazenados para a estrutura `ConfigurationData`.

O processo consiste em:

1. Abrir o arquivo de configuração.
2. Desserializar o conteúdo JSON.
3. Mapear os campos JSON para a estrutura de configuração.
4. Retornar a estrutura preenchida.

Caso ocorra erro em qualquer etapa, uma estrutura inválida é retornada (`valid = false`).

Retorna

Estrutura `ConfigurationData` contendo os dados carregados.

Documento JSON utilizado para leitura da configuração.

O tamanho foi definido como 768 bytes para acomodar todos os campos esperados no arquivo de configuração.

Indica que os dados foram carregados corretamente.

4.10.2.4 saveConfigutionData()

```
bool saveConfigutionData (
            const ConfigurationData & config)
```

Salva os dados de configuração no armazenamento persistente.

Converte os dados da estrutura `ConfigurationData` para formato JSON e grava no arquivo `/config.json` dentro do sistema de arquivos LittleFS.

Caso o arquivo já exista, ele será sobrescrito.

Parâmetros

config	Estrutura contendo os dados de configuração.

Retorna

true Se os dados foram salvos corretamente.

false Caso ocorra erro ao gravar o arquivo.

Salva os dados de configuração no armazenamento persistente.

Essa função cria ou sobrescreve o arquivo `/config.json` no sistema de arquivos LittleFS, armazenando os dados contidos na estrutura `ConfigurationData`.

O processo consiste em:

1. Converter a estrutura de configuração para JSON.
2. Abrir o arquivo em modo de escrita.
3. Serializar o JSON no arquivo.
4. Recarregar a configuração salva na variável global `cfg`.

O uso de `serializeJsonPretty()` facilita a leitura manual do arquivo durante depuração.

Parâmetros

config	Estrutura contendo os dados de configuração.

Retorna

true Se os dados foram salvos corretamente.

false Caso ocorra erro ao abrir ou escrever o arquivo.

Abre o arquivo de configuração em modo escrita.

Caso o arquivo já exista, ele será sobrescrito.

Recarrega a configuração para manter a variável global `cfg` sincronizada com o conteúdo salvo.

4.10.3 Variáveis

4.10.3.1 cfg

`ConfigurationData cfg [extern]`

Estrutura global contendo a configuração atual do sistema.

Esta variável armazena os dados de configuração atualmente utilizados pelo sistema.

Ela é atualizada quando:

- O sistema carrega a configuração do armazenamento
- Uma nova configuração é salva

Outros módulos podem acessar essa variável para obter os parâmetros de configuração ativos.

Essa variável mantém os dados de configuração carregados do arquivo `/config.json`.

Ela é atualizada quando:

- O sistema carrega a configuração do armazenamento
- Uma nova configuração é salva no dispositivo

A estrutura `ConfigurationData` contém parâmetros como:

- Credenciais WiFi
- Configurações do broker MQTT
- Identificação do dispositivo

4.11 config_storage.h

Ir para a documentação desse arquivo.

```
00001
00032
00033 #ifndef CONFIG_STORAGE
00034 #define CONFIG_STORAGE
00035
00036 #include <Arduino.h>
00037
00038
00039
00053 struct ConfigurationData {
00054
00056   String mqttServer;
00057
00059   String mqttUser;
00060
00062   String mqttPass;
00063
00065   String mqttDeviceId;
00066
00068   int mqttPort;
00069
00071   String wifiPass;
00072
```

```
  String wifiSSID;

  bool valid = false;
};


void initConfigStorage();


ConfigurationData loadConfig();


bool saveConfigutionData(const ConfigurationData &config);


bool hasConfigData();


extern ConfigurationData cfg;


#endif
```

4.12 Referência do Arquivo microcontrolador_central/display_LCD-2004_I2C.cpp

Implementação do módulo de controle do display LCD 20x4 via I2C.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "display_LCD-2004_I2C.h"
```

Funções

- LiquidCrystal_I2C lcd (I2C_ADDR, LCD_COLUMNS, LCD_LINES)

  Instância do objeto responsável pelo controle do LCD.
- void clearLine (uint8_t column, uint8_t line, uint8_t count)

  Limpa parcialmente uma linha do display.
- void fixed_data ()

  Exibe os elementos fixos da interface principal.
- void showFailureMessage (const char *msgType)

  Exibe mensagem de falha no sistema.
- void showExclusionConfirm (const char *infirmary)

  Exibe tela de confirmação para finalização de chamada.
- void showInfirmaryNumber (const char *infirmary, bool hasNursingCall, int total_items)

  Exibe o número da enfermaria atual e quantidade de chamados.
- void showIPAddress (const char *msg_IPAddress)

  Exibe o endereço IP obtido pelo dispositivo.
- void lcd2004_init ()

  Inicializa o display LCD.

Variáveis

- const uint8_t I2C_ADDR = 0x27

  Endereço I2C do módulo controlador do LCD.
- const uint8_t **LCD_COLUMNS** = 20

  Número de colunas do display LCD.
- const uint8_t **LCD_LINES** = 4

  Número de linhas do display LCD.
- DisplayState currentScreen = SCREEN_NONE

  Estado atual da tela exibida no display.

4.12.1 Descrição detalhada

Implementação do módulo de controle do display LCD 20x4 via I2C.

Este módulo é responsável por gerenciar a interface de exibição do sistema utilizando um display LCD 20x4 conectado através de um módulo I2C.

Funcionalidades principais:

- Inicialização do display
- Exibição de mensagens do sistema
- Indicação de falhas de comunicação (WiFi / MQTT)
- Exibição do número da enfermaria
- Exibição da quantidade de chamados pendentes
- Exibição do endereço IP do dispositivo

O módulo também mantém o controle da tela atual para evitar atualizações desnecessárias do display, reduzindo flicker e melhorando a experiência visual.

4.12.2 Funções

4.12.2.1 clearLine()

```
void clearLine (
            uint8_t column,
            uint8_t line,
            uint8_t count)
```

Limpa parcialmente uma linha do display.

Esta função substitui um número definido de caracteres por espaços em branco, permitindo apagar apenas uma parte específica da tela sem precisar limpar o display inteiro.

Parâmetros

column	Coluna inicial onde a limpeza começa.
line	Linha do display a ser limpa.
count	Quantidade de colunas a serem apagadas.

4.12.2.2 fixed_data()

```
void fixed_data ()
```

Exibe os elementos fixos da interface principal.

Esta função desenha os textos estáticos da tela principal do sistema, incluindo:

- Título da aplicação
- Separador visual
- Campo para número da enfermaria
- Campo para quantidade de chamados

4.12.2.3 lcd()

```
LiquidCrystal_I2C lcd (
            I2C_ADDR ,
            LCD_COLUMNS ,
            LCD_LINES )
```

Instância do objeto responsável pelo controle do LCD.

A biblioteca LiquidCrystal_I2C gerencia a comunicação com o display através do barramento I2C.

4.12.2.4 lcd2004_init()

```
void lcd2004_init ()
```

Inicializa o display LCD.

Inicializa o display LCD 20x4.

Realiza a inicialização do barramento I2C e configura o display LCD para operação.

Também ativa a iluminação de fundo do display.

Observação

Deve ser chamada durante a inicialização do sistema (normalmente no `setup()`).

4.12.2.5 showExclusionConfirm()

```
void showExclusionConfirm (
            const char * infirmary)
```

Exibe tela de confirmação para finalização de chamada.

Exibe tela de confirmação para finalização de chamado.

Esta tela solicita confirmação do usuário antes de remover um chamado da lista de pendências.

Parâmetros

infirmary	Número da enfermaria selecionada.

4.12.2.6 showFailureMessage()

```
void showFailureMessage (
            const char * msgType)
```

Exibe mensagem de falha no sistema.

Exibe mensagem de falha no display.

Apresenta uma mensagem de erro indicando falha na comunicação do sistema, podendo ser relacionada à conexão WiFi ou ao broker MQTT.

A função evita redesenhar a mesma tela de erro caso ela já esteja sendo exibida.

Parâmetros

msgType	Tipo de falha a ser exibida (WiFi ou MQTT).

4.12.2.7 showInfirmaryNumber()

```
void showInfirmaryNumber (
            const char * infirmary,
            bool hasNursingCall,
            int total_items)
```

Exibe o número da enfermaria atual e quantidade de chamados.

Exibe a tela principal com dados da enfermaria.

Atualiza a tela principal mostrando:

- Número da enfermaria selecionada
- Quantidade de chamados pendentes

Se existir mais de um chamado, são exibidas setas indicando possibilidade de navegação entre itens.

Parâmetros

infirmary	Número da enfermaria exibida.
hasNursingCall	Indica se existe chamado ativo.
total_items	Quantidade total de chamados pendentes.

4.12.2.8 showIPAddress()

```
void showIPAddress (
            const char * msg_IPAddress)
```

Exibe o endereço IP obtido pelo dispositivo.

Exibe o endereço IP do dispositivo.

Mostra informações da conexão WiFi juntamente com o endereço IP atual do dispositivo na rede.

Parâmetros

msg_IPAddress	String contendo o endereço IP.

4.12.3 Variáveis

4.12.3.1 currentScreen

```
DisplayState currentScreen = SCREEN_NONE
```

Estado atual da tela exibida no display.

Estado atual da interface do display.

Utilizado para evitar redesenho desnecessário da tela, atualizando apenas quando o estado muda.

4.12.3.2 I2C_ADDR

```
const uint8_t I2C_ADDR = 0x27
```

Endereço I2C do módulo controlador do LCD.

O endereço padrão da maioria dos módulos LCD I2C baseados no chip PCF8574 é 0x27, porém pode variar dependendo do fabricante do módulo.

4.13 Referência do Arquivo microcontrolador_central/display_LCD-2004_I2C.h

Interface do módulo de controle do display LCD 20x4 via I2C.

Enumerações

- enum DisplayState {
  SCREEN_NONE , SCREEN_MAIN , SCREEN_FAILURE_WIFI , SCREEN_FAILURE_MQTT , SCREEN_EXCLUSION_CONFIRM , SCREEN_IPADDRESS }

  Define os estados possíveis das telas exibidas no display.

Funções

- void lcd2004_init ()

  Inicializa o display LCD 20x4.
- void showFailureMessage (const char ∗msgType)

  Exibe mensagem de falha no display.
- void showInfirmaryNumber (const char ∗infirmary, bool hasNursingCall, int total_items)

  Exibe a tela principal com dados da enfermaria.
- void showExclusionConfirm (const char ∗infirmary)

  Exibe tela de confirmação para finalização de chamado.
- void showIPAddress (const char ∗msg_IPAddress)

  Exibe o endereço IP do dispositivo.

Variáveis

- constexpr const char ∗ MESSAGE_WIFI = "Wi-Fi"

  Identificador de falha de conexão WiFi.
- constexpr const char ∗ MESSAGE_MQTT = "MQTT"

  Identificador de falha de comunicação MQTT.
- DisplayState currentScreen

  Estado atual da interface do display.

4.13.1 Descrição detalhada

Interface do módulo de controle do display LCD 20x4 via I2C.

Este módulo fornece funções responsáveis pela interface visual do sistema utilizando um display LCD 20x4 conectado através de um módulo I2C.

O display é utilizado para apresentar informações operacionais do sistema, incluindo:

- Número da enfermaria selecionada
- Quantidade de chamados pendentes
- Mensagens de erro de comunicação (WiFi ou MQTT)
- Confirmação de ações do usuário
- Endereço IP do dispositivo na rede

O módulo também implementa uma pequena máquina de estados para controlar qual tela está sendo exibida no momento, evitando atualizações desnecessárias do display.

Observação

Este módulo depende das bibliotecas:

- Wire
- LiquidCrystal_I2C

Data

2026

4.13.2 Enumerações

4.13.2.1 DisplayState

`enum DisplayState`

Define os estados possíveis das telas exibidas no display.

Essa enumeração representa a máquina de estados responsável por controlar qual tela está atualmente ativa no display.

O uso dessa estrutura permite evitar redesenhos desnecessários da tela e facilita o gerenciamento da interface do usuário.

Enumeradores

SCREEN_NONE	Nenhuma tela ativa
SCREEN_MAIN	Tela principal com número da enfermaria e quantidade de chamados
SCREEN_FAILURE_WIFI	Tela de falha na conexão WiFi. Esta tela é exibida apenas uma vez enquanto a falha persistir.
SCREEN_FAILURE_MQTT	Tela de falha na comunicação MQTT. Esta tela também é exibida apenas uma vez enquanto o erro persistir.
SCREEN_EXCLUSION_CONFIRM	Tela de confirmação para finalizar um chamado
SCREEN_IPADDRESS	Tela que exibe o endereço IP do dispositivo

4.13.3 Funções

4.13.3.1 lcd2004_init()

`void lcd2004_init () [extern]`

Inicializa o display LCD 20x4.

Esta função inicializa a comunicação I2C e prepara o display para operação.

Durante a inicialização:

- O barramento I2C é iniciado
- O display é configurado
- A iluminação de fundo é ativada

Observação

Deve ser chamada durante o `setup()` do sistema.

Inicializa o display LCD 20x4.

Realiza a inicialização do barramento I2C e configura o display LCD para operação.

Também ativa a iluminação de fundo do display.

Observação

Deve ser chamada durante a inicialização do sistema (normalmente no `setup()`).

4.13.3.2 showExclusionConfirm()

```
void showExclusionConfirm (
            const char * infirmary)  [extern]
```

Exibe tela de confirmação para finalização de chamado.

Solicita confirmação do usuário antes de remover um chamado da lista de pendências.

Parâmetros

infirmary	Número da enfermaria selecionada.

Exibe tela de confirmação para finalização de chamado.

Esta tela solicita confirmação do usuário antes de remover um chamado da lista de pendências.

Parâmetros

infirmary	Número da enfermaria selecionada.

4.13.3.3 showFailureMessage()

```
void showFailureMessage (
            const char * msgType)  [extern]
```

Exibe mensagem de falha no display.

Apresenta uma mensagem informando falha de comunicação no sistema.

Os tipos de falha suportados são:

- MESSAGE_WIFI
- MESSAGE_MQTT

Parâmetros

msgType	Tipo da falha a ser exibida.

Exibe mensagem de falha no display.

Apresenta uma mensagem de erro indicando falha na comunicação do sistema, podendo ser relacionada à conexão WiFi ou ao broker MQTT.

A função evita redesenhar a mesma tela de erro caso ela já esteja sendo exibida.

Parâmetros

msgType	Tipo de falha a ser exibida (WiFi ou MQTT).

4.13.3.4 showInfirmaryNumber()

```
void showInfirmaryNumber (
            const char * infirmary,
            bool hasNursingCall,
            int total_items)  [extern]
```

Exibe a tela principal com dados da enfermaria.

Mostra no display:

- Número da enfermaria selecionada
- Quantidade de chamados pendentes

Caso existam múltiplos chamados, setas de navegação são exibidas ao lado do número da enfermaria.

Parâmetros

infirmary	Número da enfermaria.
hasNursingCall	Indica se existe chamado ativo.
total_items	Quantidade total de chamados pendentes.

Exibe a tela principal com dados da enfermaria.

Atualiza a tela principal mostrando:

- Número da enfermaria selecionada
- Quantidade de chamados pendentes

Se existir mais de um chamado, são exibidas setas indicando possibilidade de navegação entre itens.

Parâmetros

infirmary	Número da enfermaria exibida.
hasNursingCall	Indica se existe chamado ativo.
total_items	Quantidade total de chamados pendentes.

4.13.3.5 showIPAddress()

```
void showIPAddress (
            const char * msg_IPAddress)  [extern]
```

Exibe o endereço IP do dispositivo.

Mostra informações da conexão WiFi juntamente com o endereço IP obtido pelo dispositivo na rede.

Parâmetros

msg_IPAddress	Endereço IP do dispositivo.

Exibe o endereço IP do dispositivo.

Mostra informações da conexão WiFi juntamente com o endereço IP atual do dispositivo na rede.

Parâmetros

msg_IPAddress	String contendo o endereço IP.

4.13.4 Variáveis

4.13.4.1 currentScreen

```
DisplayState currentScreen  [extern]
```

Estado atual da interface do display.

Essa variável mantém o estado da tela atualmente exibida. Ela é utilizada para controlar a máquina de estados da interface gráfica e evitar atualizações desnecessárias no display.

Estado atual da interface do display.

Utilizado para evitar redesenho desnecessário da tela, atualizando apenas quando o estado muda.

4.13.4.2 MESSAGE_MQTT

```
const char* MESSAGE_MQTT = "MQTT"  [constexpr]
```

Identificador de falha de comunicação MQTT.

Utilizado pela função `showFailureMessage()` para indicar que ocorreu erro na comunicação com o broker MQTT.

4.13.4.3 MESSAGE_WIFI

```
const char* MESSAGE_WIFI = "Wi-Fi"  [constexpr]
```

Identificador de falha de conexão WiFi.

Utilizado pela função `showFailureMessage()` para indicar que ocorreu erro na conexão com a rede WiFi.

4.14 display_LCD-2004_I2C.h

Ir para a documentação desse arquivo.

```
00001
00029
00030 #ifndef DISPLAY_LCD_2004_I2C_H // Verifica se o cabeçalho já foi incluído
00031 #define DISPLAY_LCD_2004_I2C_H // Define o identificador do cabeçalho para evitar inclusões múltiplas
00032
00033
00040 constexpr const char* MESSAGE_WIFI = "Wi-Fi";
00041
00048 constexpr const char* MESSAGE_MQTT = "MQTT";
00049
00050
00051 // ------------------------------------------------------------------------------
00052 // Máquina de estados do display
00053 // ------------------------------------------------------------------------------
00054
00066 enum DisplayState {
00067
00069     SCREEN_NONE,
00070
00072     SCREEN_MAIN,
00073
00079     SCREEN_FAILURE_WIFI,
00080
00086     SCREEN_FAILURE_MQTT,
00087
00089     SCREEN_EXCLUSION_CONFIRM,
00090
00092     SCREEN_IPADDRESS
00093 };
00094
00104 extern DisplayState currentScreen;
00105
00106 // ------------------------------------------------------------------------------
00107 // Funções públicas do módulo
00108 // ------------------------------------------------------------------------------
00109
00124 extern void lcd2004_init();
00125
00140 extern void showFailureMessage(const char* msgType);
00141
00158 extern void showInfirmaryNumber(const char* infirmary, bool hasNursingCall, int total_items);
00159
00169 extern void showExclusionConfirm(const char* infirmary);
00170
00180 extern void showIPAddress(const char* msg_IPAddress);
00181
00182
00183 #endif // Final da diretiva de proteção contra múltiplas inclusões
```

4.15 Referência do Arquivo microcontrolador_central/DoublyLinkedList_NursingCall.cpp

Implementação de uma lista duplamente encadeada para gerenciamento de chamadas de enfermagem.

```
#include "DoublyLinkedList_NursingCall.h"
#include <cstring>
```

4.15.1 Descrição detalhada

Implementação de uma lista duplamente encadeada para gerenciamento de chamadas de enfermagem.

Este módulo implementa uma estrutura de dados baseada em lista duplamente encadeada para armazenar chamadas de enfermagem pendentes.

Cada nó da lista representa um chamado contendo:

- Número da enfermaria
- Identificador do dispositivo

Funcionalidades principais:

- Inserção de novos chamados
- Navegação entre chamados
- Remoção de chamados
- Remoção automática quando atingir limite de memória
- Busca e remoção por ID
- Limpeza completa da lista

A lista mantém ponteiros para:

- Primeiro elemento (`head`)
- Último elemento (`tail`)
- Elemento atual (`current`)

Isso permite navegação eficiente entre os nós.

4.16 DoublyLinkedList_NursingCall.h

```
00001 #ifndef DOUBLY_LINKED_LIST_NURSING_CALL_H
00002 #define DOUBLY_LINKED_LIST_NURSING_CALL_H
00003
00028
00029
00030 // ---------------------------------------------------------------------------
00031 // Estrutura de nó da lista
00032 // ---------------------------------------------------------------------------
00033
00043 struct NursingCall {
00044
00046   char infirmary[32];
00047
00049   char id[32];
00050
00052   NursingCall* next;
00053
00055   NursingCall* prev;
00056 };
00057
00058
00059 // ---------------------------------------------------------------------------
00060 // Classe de gerenciamento da lista
00061 // ---------------------------------------------------------------------------
00062
00078 class List_NursingCall {
00079
00080   private:
00081
00083     NursingCall* head;
00084
00086     NursingCall* tail;
00087
00093     NursingCall* current;
00094
00096     int total;
00097
00104     static const int MAX_CALLS = 300;
00105
00116     bool doNotRemoveCurrent = false;
```

```
00117
00118
00126     bool isValidInfirmary(const char* infirmary);
00127
00128
00139     bool removeOldestCall();
00140
00141
00149     bool remove(NursingCall* node);
00150
00151
00152   public:
00153
00159     List_NursingCall();
00160
00161
00167     ~List_NursingCall();
00168
00169
00170     // ---------------------------------------------------------------------------
00171     // Consultas
00172     // ---------------------------------------------------------------------------
00173
00179     int getTotal();
00180
00181
00188     const char* getInfirmaryCurrent();
00189
00190
00197     const char* getIdCurrent();
00198
00199
00206     bool hasNursingCall();
00207
00208
00216     void setDoNotRemoveCurrent(bool value);
00217
00218
00224     bool getDoNotRemoveCurrent() const;
00225
00226
00227     // ---------------------------------------------------------------------------
00228     // Navegação
00229     // ---------------------------------------------------------------------------
00230
00236     void next();
00237
00238
00244     void prev();
00245
00246
00247     // ---------------------------------------------------------------------------
00248     // Modificações
00249     // ---------------------------------------------------------------------------
00250
00260     bool add(const char* infirmary, const char* id);
00261
00262
00269     bool removeCurrent();
00270
00271
00279     bool removalById(const char* id);
00280
00281
00287     void clear();
00288 };
00289
00290 #endif  // DOUBLY_LINKED_LIST_NURSING_CALL_H
```

4.17 Referência do Arquivo microcontrolador_central/jsonDataProcessing.cpp

Processamento de mensagens JSON recebidas e enviadas via MQTT.

```
#include "jsonDataProcessing.h"
#include "listNursingCall_utils.h"
#include <ArduinoJson.h>
#include <cstring>
```

```
#include "log.h"
#include "config.h"
#include "config_storage.h"
```

Funções

- bool processing_json_MQTT (byte ∗payload, unsigned int length)

  Processa um payload JSON recebido via MQTT.
- const char ∗ createJsonPayload (char ∗buffer, size_t length, const char ∗roomNumber)

  Cria payload JSON para envio via MQTT.
- const char ∗ getPayloadID (byte ∗payload, unsigned int length)

  Extrai apenas o campo "id" de um payload JSON.
- const char ∗ creteJsonPayloadConfirmationMessage (char ∗buffer, size_t length)

  Cria payload JSON de confirmação de mensagem.

4.17.1 Descrição detalhada

Processamento de mensagens JSON recebidas e enviadas via MQTT.

Este módulo é responsável por:

- Interpretar mensagens JSON recebidas pelo broker MQTT
- Extrair campos relevantes das mensagens
- Inserir novas chamadas na lista de chamadas de enfermaria
- Criar mensagens JSON para publicação MQTT

A biblioteca utilizada para manipulação de JSON é a ArduinoJson, que permite parse e serialização de JSON de forma eficiente em sistemas embarcados.

4.17.2 Funções

4.17.2.1 createJsonPayload()

```
const char * createJsonPayload (
            char * buffer,
            size_t length,
            const char * roomNumber)
```

Cria payload JSON para envio via MQTT.

Cria um payload JSON para envio via MQTT.

Esta função gera uma mensagem JSON contendo informações do dispositivo que está respondendo à chamada.

O JSON gerado possui o formato:

{ "id": "<device_id>", "estado": "ocioso", "mensagem": "Desligar LED", "room_number": "...", "local": "posto_↩ enfermaria", "comando": "desligar" }

Parâmetros

buffer	Buffer onde o JSON será armazenado.
length	Tamanho máximo do buffer.
roomNumber	Número da enfermaria relacionada à mensagem.

Retorna

Ponteiro para o buffer contendo o JSON serializado.

4.17.2.2 creteJsonPayloadConfirmationMessage()

```
const char * creteJsonPayloadConfirmationMessage (
            char * buffer,
            size_t length)
```

Cria payload JSON de confirmação de mensagem.

Cria um payload JSON de confirmação de mensagem.

Utilizado para confirmar o recebimento de uma mensagem MQTT.

JSON gerado:

{ "status": "ok" }

Parâmetros

buffer	Buffer onde o JSON será armazenado.
length	Tamanho máximo do buffer.

Retorna

Ponteiro para o buffer contendo o JSON serializado.

4.17.2.3 getPayloadID()

```
const char * getPayloadID (
            byte * payload,
            unsigned int length)
```

Extrai apenas o campo "id" de um payload JSON.

Extrai o campo "id" de um payload JSON.

Utilizado quando é necessário identificar rapidamente qual dispositivo enviou uma mensagem MQTT.

Parâmetros

payload	Dados recebidos do MQTT.
length	Tamanho do payload.

Retorna

Ponteiro para string contendo o ID extraído ou mensagem de erro caso o parse falhe.

4.17.2.4 processing_json_MQTT()

```
bool processing_json_MQTT (
            byte * payload,
            unsigned int length)
```

Processa um payload JSON recebido via MQTT.

Esta função:

1. Converte o payload recebido em um objeto JSON.
2. Valida os campos obrigatórios do JSON.
3. Extrai as informações relevantes.
4. Insere a chamada na lista de chamadas de enfermaria.

Caso ocorra erro no parse ou em algum campo obrigatório, a função retorna `false`.

Parâmetros

payload	Ponteiro para os dados recebidos pelo MQTT.
length	Tamanho do payload recebido.

Retorna

true se o processamento foi realizado com sucesso.
false caso ocorra erro.

Observação

Conversão temporária para String. Utiliza variável estática para manter o buffer válido após a função.

4.18 Referência do Arquivo microcontrolador_central/jsonDataProcessing.h

Interface para processamento de mensagens JSON utilizadas na comunicação MQTT.

```
#include <Arduino.h>
```

Funções

- bool processing_json_MQTT (byte ∗payload, unsigned int length)

  Processa um payload JSON recebido via MQTT.
- const char ∗ createJsonPayload (char ∗buffer, size_t length, const char ∗roomNumber)

  Cria um payload JSON para envio via MQTT.
- const char ∗ getPayloadID (byte ∗payload, unsigned int length)

  Extrai o campo "id" de um payload JSON.
- const char ∗ creteJsonPayloadConfirmationMessage (char ∗buffer, size_t length)

  Cria um payload JSON de confirmação de mensagem.

4.18.1 Descrição detalhada

Interface para processamento de mensagens JSON utilizadas na comunicação MQTT.

Este módulo define as funções responsáveis por:

- Interpretar mensagens JSON recebidas via MQTT
- Extrair informações específicas de payloads JSON
- Criar mensagens JSON para publicação no broker MQTT
- Gerar mensagens de confirmação de recebimento

A implementação utiliza a biblioteca **ArduinoJson** para realizar a serialização e desserialização dos dados JSON.

As funções deste módulo são utilizadas principalmente na camada de comunicação do sistema, intermediando os dados recebidos e enviados pelo broker MQTT.

4.18.2 Funções

4.18.2.1 createJsonPayload()

```
const char * createJsonPayload (
            char * buffer,
            size_t length,
            const char * roomNumber)
```

Cria um payload JSON para envio via MQTT.

Esta função gera um JSON contendo informações do dispositivo e da enfermaria associada a uma ação.

O JSON gerado normalmente é utilizado para indicar o estado atual do dispositivo e enviar comandos ou respostas para o broker MQTT.

Parâmetros

buffer	Buffer onde o JSON serializado será armazenado.
length	Tamanho máximo do buffer.

roomNumber	Número da enfermaria associado à mensagem.

Retorna

Ponteiro para o buffer contendo o JSON serializado.

Cria um payload JSON para envio via MQTT.

Esta função gera uma mensagem JSON contendo informações do dispositivo que está respondendo à chamada.

O JSON gerado possui o formato:

{ "id": "<device_id>", "estado": "ocioso", "mensagem": "Desligar LED", "room_number": "...", "local": "posto_↩ enfermaria", "comando": "desligar" }

Parâmetros

buffer	Buffer onde o JSON será armazenado.
length	Tamanho máximo do buffer.
roomNumber	Número da enfermaria relacionada à mensagem.

Retorna

Ponteiro para o buffer contendo o JSON serializado.

4.18.2.2 creteJsonPayloadConfirmationMessage()

```
const char * creteJsonPayloadConfirmationMessage (
            char * buffer,
            size_t length)
```

Cria um payload JSON de confirmação de mensagem.

Esta função gera um JSON simples indicando que uma mensagem foi recebida e processada com sucesso.

Exemplo de JSON gerado:

{ "status": "ok" }

Esse payload pode ser enviado para um tópico MQTT de confirmação.

Parâmetros

buffer	Buffer onde o JSON será armazenado.
length	Tamanho máximo do buffer.

Retorna

Ponteiro para o buffer contendo o JSON serializado.

Cria um payload JSON de confirmação de mensagem.

Utilizado para confirmar o recebimento de uma mensagem MQTT.

JSON gerado:

{ "status": "ok" }

Parâmetros

buffer	Buffer onde o JSON será armazenado.
length	Tamanho máximo do buffer.

Retorna

Ponteiro para o buffer contendo o JSON serializado.

4.18.2.3 getPayloadID()

```
const char * getPayloadID (
            byte * payload,
            unsigned int length)
```

Extrai o campo "id" de um payload JSON.

Utilizado quando é necessário identificar rapidamente qual dispositivo ou entidade enviou uma mensagem MQTT.

A função realiza o parse do JSON e retorna o valor do campo `"id"`.

Parâmetros

payload	Ponteiro para os dados recebidos via MQTT.
length	Tamanho do payload.

Retorna

Ponteiro para string contendo o ID extraído do JSON.

Extrai o campo "id" de um payload JSON.

Utilizado quando é necessário identificar rapidamente qual dispositivo enviou uma mensagem MQTT.

Parâmetros

payload	Dados recebidos do MQTT.

length	Tamanho do payload.

Retorna

Ponteiro para string contendo o ID extraído ou mensagem de erro caso o parse falhe.

4.18.2.4 processing_json_MQTT()

```
bool processing_json_MQTT (
            byte * payload,
            unsigned int length)  [extern]
```

Processa um payload JSON recebido via MQTT.

Esta função interpreta um payload recebido do broker MQTT, realiza o parse do JSON e extrai os campos necessários para registrar uma nova chamada de enfermaria no sistema.

Caso o JSON esteja inválido ou algum campo obrigatório esteja ausente, a função retorna `false`.

Parâmetros

payload	Ponteiro para os dados brutos recebidos via MQTT.
length	Tamanho do payload recebido.

Retorna

true se o JSON foi processado corretamente.

false caso ocorra erro no parse ou nos dados.

Esta função:

1. Converte o payload recebido em um objeto JSON.
2. Valida os campos obrigatórios do JSON.
3. Extrai as informações relevantes.
4. Insere a chamada na lista de chamadas de enfermaria.

Caso ocorra erro no parse ou em algum campo obrigatório, a função retorna `false`.

Parâmetros

payload	Ponteiro para os dados recebidos pelo MQTT.
length	Tamanho do payload recebido.

Retorna

true se o processamento foi realizado com sucesso.

false caso ocorra erro.

Observação

Conversão temporária para String. Utiliza variável estática para manter o buffer válido após a função.

4.19 jsonDataProcessing.h

Ir para a documentação desse arquivo.

```
00001
00020
00021 #ifndef JSONDATAPROCESSING_H
00022 #define JSONDATAPROCESSING_H
00023
00024 #include <Arduino.h>
00025
00026
00027
00045 extern bool processing_json_MQTT(byte* payload, unsigned int length);
00046
00047
00065 const char* createJsonPayload(char* buffer, size_t length, const char* roomNumber);
00066
00067
00083 const char* getPayloadID(byte* payload, unsigned int length);
00084
00085
00106 const char* creteJsonPayloadConfirmationMessage(char* buffer, size_t length);
00107
00108 #endif
```

4.20 Referência do Arquivo microcontrolador_central/led.cpp

Implementação do controle do LED indicador do sistema.

```
#include "led.h"
#include "config.h"
```

Funções

- void ledInit ()

  Inicializa o LED configurando o pino como saída.
- void toggleLed ()

  Alterna o estado do LED em intervalos regulares.
- void turnOffLed ()

  Desliga o LED forçando o pino para nível lógico LOW.
- void turnOnLed ()

  Liga o LED forçando o pino para nível lógico HIGH.

Variáveis

- unsigned long lastTimeLedTurnedOn = 0

  Armazena o momento (em ms) em que o LED foi alternado pela última vez.
- constexpr unsigned long LED_INTERVAL_MS = 1000

  Intervalo em milissegundos para alternar o estado do LED.

4.20.1 Descrição detalhada

Implementação do controle do LED indicador do sistema.

Este módulo é responsável por gerenciar o LED conectado ao microcontrolador, permitindo sua inicialização, acionamento, desligamento e operação em modo intermitente (piscar).

O controle do piscar do LED é realizado utilizando a função `millis()`, evitando o uso de `delay()` e permitindo que o sistema continue executando outras tarefas simultaneamente. Esse método é conhecido como **temporização não bloqueante**, sendo amplamente utilizado em sistemas embarcados baseados em Arduino e ESP.

Funcionalidades implementadas neste módulo:

- Inicialização do pino do LED
- Acionamento manual do LED (ligar/desligar)
- Alternância automática do estado do LED em intervalos definidos

O pino utilizado para controle do LED é definido no arquivo `config.h`.

Observação

Este módulo depende das seguintes configurações:

- `PIN_LED` definido em `config.h`

Veja também

config.h

Data

2026

4.20.2 Funções

4.20.2.1 ledInit()

```
void ledInit ()
```

Inicializa o LED configurando o pino como saída.

Deve ser chamada no setup() para preparar o hardware antes do uso.

4.20.2.2 toggleLed()

```
void toggleLed ()
```

Alterna o estado do LED em intervalos regulares.

Utiliza a função millis() para verificar se já se passou o tempo definido em LED_INTERVAL_MS desde a última alteração de estado.

Parâmetros

now	- Tempo atual em milissegundos (retornado por millis()).

4.20.2.3 turnOffLed()

```
void turnOffLed ()
```

Desliga o LED forçando o pino para nível lógico LOW.

Desliga o LED (força nível lógico LOW no pino).

4.20.2.4 turnOnLed()

```
void turnOnLed ()
```

Liga o LED forçando o pino para nível lógico HIGH.

Liga o LED (força nível lógico HIGH no pino).

4.20.3 Variáveis

4.20.3.1 lastTimeLedTurnedOn

```
unsigned long lastTimeLedTurnedOn = 0
```

Armazena o momento (em ms) em que o LED foi alternado pela última vez.

Usado junto com a função millis() para controlar o intervalo de piscar o LED sem bloquear o código com delays.

4.20.3.2 LED_INTERVAL_MS

```
unsigned long LED_INTERVAL_MS = 1000  [constexpr]
```

Intervalo em milissegundos para alternar o estado do LED.

Neste caso, definido como 1000 ms (1 segundos).

4.21 Referência do Arquivo microcontrolador_central/led.h

Interface para controle do LED indicador do sistema.

```
#include <Arduino.h>
```

Funções

- void ledInit ()

  Inicializa o LED configurando o pino como saída.
- void toggleLed ()

  Alterna o estado do LED em intervalos regulares.
- void turnOnLed ()

  Liga o LED (força nível lógico HIGH no pino).
- void turnOffLed ()

  Desliga o LED (força nível lógico LOW no pino).

4.21.1 Descrição detalhada

Interface para controle do LED indicador do sistema.

Este módulo define a interface pública para controle do LED conectado ao microcontrolador. Ele fornece funções simples para inicialização do hardware, acionamento direto do LED e operação em modo intermitente (piscar).

O comportamento de piscar do LED é implementado no arquivo `led.cpp` utilizando temporização baseada na função `millis()`, evitando o uso de `delay()` e permitindo execução não bloqueante do firmware.

Funcionalidades disponíveis:

- Inicializar o pino do LED
- Ligar o LED
- Desligar o LED
- Alternar o estado do LED periodicamente

O pino utilizado para controle do LED é definido no arquivo `config.h`

Veja também

led.cpp

config.h

Data

2026

4.21.2 Funções

4.21.2.1 ledInit()

```
void ledInit ()
```

Inicializa o LED configurando o pino como saída.

Deve ser chamada uma vez no setup() para preparar o hardware.

Deve ser chamada no setup() para preparar o hardware antes do uso.

4.21.2.2 toggleLed()

```
void toggleLed ()
```

Alterna o estado do LED em intervalos regulares.

A lógica de controle do tempo utiliza millis() para evitar delays bloqueantes.

Utiliza a função millis() para verificar se já se passou o tempo definido em LED_INTERVAL_MS desde a última alteração de estado.

Parâmetros

now	- Tempo atual em milissegundos (retornado por millis()).

4.21.2.3 turnOffLed()

```
void turnOffLed ()
```

Desliga o LED (força nível lógico LOW no pino).

Desliga o LED (força nível lógico LOW no pino).

4.21.2.4 turnOnLed()

```
void turnOnLed ()
```

Liga o LED (força nível lógico HIGH no pino).

Liga o LED (força nível lógico HIGH no pino).

4.22 led.h

Ir para a documentação desse arquivo.

```
00001
00027 #ifndef LED_H
00028 #define LED_H
00029
00030 #include <Arduino.h>
00031
00037 void ledInit();
00038
00045 void toggleLed();
00046
00050 void turnOnLed();
00051
00055 void turnOffLed();
00056
00057 #endif
```

4.23 Referência do Arquivo microcontrolador_central/listNursingCall_utils.cpp

Definição das variáveis globais utilizadas no gerenciamento da lista de chamadas de enfermaria.

`#include "listNursingCall_utils.h"`

Variáveis

- List_NursingCall listCalls

  Instância global da lista de chamadas de enfermaria.
- volatile bool listUpdated = false

  Flag que indica atualização na lista de chamadas.

4.23.1 Descrição detalhada

Definição das variáveis globais utilizadas no gerenciamento da lista de chamadas de enfermaria.

Este módulo define instâncias globais utilizadas para o gerenciamento das chamadas de enfermaria no sistema.

A lista de chamadas é implementada através da classe `List_NursingCall`, que utiliza uma estrutura de **lista duplamente encadeada** para armazenar e navegar entre as chamadas recebidas.

Além da lista principal, também é utilizada uma **flag de atualização** que indica quando houve alteração na lista, permitindo que outros módulos (como o display) saibam quando precisam atualizar as informações exibidas.

Esse mecanismo evita atualizações desnecessárias do display, melhorando a eficiência do sistema.

Veja também

List_NursingCall

listNursingCall_utils.h

Data

2026

4.23.2 Variáveis

4.23.2.1 listCalls

```
List_NursingCall listCalls
```

Instância global da lista de chamadas de enfermaria.

Estrutura de lista duplamente encadeada utilizada para armazenar chamadas de enfermagem.

Esta estrutura armazena todas as chamadas recebidas pelo sistema, permitindo inserção, remoção e navegação entre os registros.

A lista é manipulada por diferentes módulos do sistema, como:

- Processamento de mensagens MQTT
- Interface de usuário (display)

4.23.2.2 listUpdated

```
volatile bool listUpdated = false
```

Flag que indica atualização na lista de chamadas.

Esta variável é utilizada para sinalizar que a lista de chamadas foi modificada (inserção ou remoção).

Quando essa flag é ativada (`true`), o módulo responsável pela interface do usuário (ex.: display LCD) pode atualizar as informações exibidas.

A variável é declarada como `volatile` para garantir que o compilador não realize otimizações indevidas, já que seu valor pode ser alterado por diferentes partes do firmware.

4.24 Referência do Arquivo microcontrolador_central/listNursingCall_utils.h

Declarações globais para gerenciamento da lista de chamadas de enfermaria.

```
#include "DoublyLinkedList_NursingCall.h"
```

Variáveis

- List_NursingCall listCalls

  Estrutura de lista duplamente encadeada utilizada para armazenar chamadas de enfermagem.
- volatile bool listUpdated

  Flag que indica atualização na lista de chamadas.

4.24.1 Descrição detalhada

Declarações globais para gerenciamento da lista de chamadas de enfermaria.

Este módulo fornece acesso global à estrutura responsável por armazenar as chamadas de enfermagem recebidas pelo sistema.

A lista é implementada por meio de uma **lista duplamente encadeada** definida em `DoublyLinkedList_NursingCall.h`. Essa estrutura permite:

- Inserção de novas chamadas
- Remoção de chamadas
- Navegação entre chamadas (anterior/próxima)
- Consulta do número total de chamadas

Além da lista principal, o módulo também expõe uma **flag de atualização** utilizada para indicar que a lista sofreu alguma modificação.

Essa flag é usada principalmente pelo módulo de interface (ex.: display) para evitar atualizações desnecessárias da tela, atualizando apenas quando ocorre alguma alteração na lista.

Observação

As variáveis declaradas neste arquivo são definidas no arquivo `listNursingCall_utils.cpp`.

Veja também

DoublyLinkedList_NursingCall.h

listNursingCall_utils.cpp

Autor

Fabrício Andrade Sousa

Data

2026

4.24.2 Variáveis

4.24.2.1 listCalls

```
List_NursingCall listCalls [extern]
```

Estrutura de lista duplamente encadeada utilizada para armazenar chamadas de enfermagem.

Instância global da lista de chamadas de enfermaria.

Esta estrutura armazena todas as chamadas recebidas pelo sistema. Ela pode ser acessada por diferentes módulos do firmware, como:

- Processamento de mensagens MQTT
- Interface do usuário (display)
- Lógica de navegação entre chamadas

Estrutura de lista duplamente encadeada utilizada para armazenar chamadas de enfermagem.

Esta estrutura armazena todas as chamadas recebidas pelo sistema, permitindo inserção, remoção e navegação entre os registros.

A lista é manipulada por diferentes módulos do sistema, como:

- Processamento de mensagens MQTT
- Interface de usuário (display)

4.24.2.2 listUpdated

```
volatile bool listUpdated [extern]
```

Flag que indica atualização na lista de chamadas.

Essa variável é utilizada para sinalizar que ocorreu alguma modificação na lista de chamadas (inserção ou remoção).

Quando essa flag é definida como `true`, módulos como o display podem atualizar a interface do usuário para refletir as novas informações.

A palavra-chave `volatile` é utilizada para garantir que o compilador não otimize o acesso à variável, permitindo que mudanças realizadas em diferentes partes do firmware sejam sempre detectadas corretamente.

Esta variável é utilizada para sinalizar que a lista de chamadas foi modificada (inserção ou remoção).

Quando essa flag é ativada (`true`), o módulo responsável pela interface do usuário (ex.: display LCD) pode atualizar as informações exibidas.

A variável é declarada como `volatile` para garantir que o compilador não realize otimizações indevidas, já que seu valor pode ser alterado por diferentes partes do firmware.

4.25 listNursingCall_utils.h

Ir para a documentação desse arquivo.

```
#ifndef LISTNURSINGCALL_UTILS_H
#define LISTNURSINGCALL_UTILS_H


// ---------------------------------------------------------------------------
// Dependências
// ---------------------------------------------------------------------------

#include "DoublyLinkedList_NursingCall.h"


// ---------------------------------------------------------------------------
// Variáveis globais
// ---------------------------------------------------------------------------

extern List_NursingCall listCalls;


extern volatile bool listUpdated;


#endif
```

4.26 Referência do Arquivo microcontrolador_central/log.cpp

Implementação de funções de logging para depuração e monitoramento do sistema.

```
#include "log.h"
#include <Arduino.h>
#include <stdarg.h>
```

Funções

- void logInit (LogLevel level)

  Inicializa o sistema de logs.
- void log (LogLevel level, const char *fmt,...)

  Registra uma mensagem de log com formato customizado.

Variáveis

- LogLevel CURRENT_LOG_LEVEL = LOG_DEBUG

  Variável global que armazena o nível de log atual.

4.26.1 Descrição detalhada

Implementação de funções de logging para depuração e monitoramento do sistema.

Este módulo fornece um mecanismo centralizado para registrar mensagens no Serial em diferentes níveis de severidade (ERRO, WARN, INFO, DEBUG).

4.26.2 Funções

4.26.2.1 log()

```
void log (
            LogLevel level,
            const char * fmt,
             ...)
```

Registra uma mensagem de log com formato customizado.

Registra uma mensagem no log.

Esta função formata a mensagem recebida e imprime na Serial, respeitando o nível atual de log definido.

Parâmetros

level	Nível da mensagem (ERRO, WARN, INFO, DEBUG).
fmt	String de formato (similar ao printf).
...	Argumentos variáveis correspondentes ao formato.

4.26.2.2 logInit()

```
void logInit (
            LogLevel level)
```

Inicializa o sistema de logs.

Inicializa o sistema de log.

- Define o nível de log inicial.
- Inicia a comunicação Serial a 115200 baud.
- Registra uma mensagem indicando que o log foi iniciado.

Parâmetros

level	Nível de log a ser utilizado (LOG_ERROR, LOG_WARN, LOG_INFO, LOG_DEBUG).

4.26.3 Variáveis

4.26.3.1 CURRENT_LOG_LEVEL

```
LogLevel CURRENT_LOG_LEVEL = LOG_DEBUG
```

Variável global que armazena o nível de log atual.

Controla quais mensagens serão exibidas. Deve ser inicializada pela função logInit().

4.27 Referência do Arquivo microcontrolador_central/log.h

Declarações para o sistema de logging do projeto.

```
#include <Arduino.h>
#include <stdarg.h>
```

Enumerações

- enum LogLevel {
  LOG_NONE = 0 , LOG_ERROR = 1 , LOG_WARN = 2 , LOG_INFO = 3 ,
  LOG_DEBUG = 4 }

  Define os níveis de log disponíveis.

Funções

- void logInit (LogLevel level)

  Inicializa o sistema de log.
- void log (LogLevel level, const char *fmt,...)

  Registra uma mensagem no log.

Variáveis

- LogLevel CURRENT_LOG_LEVEL

  Variável global que armazena o nível de log atual.

4.27.1 Descrição detalhada

Declarações para o sistema de logging do projeto.

Este módulo define níveis de log e funções utilitárias para registrar mensagens em diferentes níveis de severidade. Pode ser configurado para controlar a quantidade de informações enviadas à saída serial durante a execução.

4.27.2 Enumerações

4.27.2.1 LogLevel

```
enum LogLevel
```

Define os níveis de log disponíveis.

Enumeradores

LOG_NONE	Nenhum log será exibido
LOG_ERROR	Mensagens de erro (alta prioridade)

LOG_WARN	Mensagens de aviso
LOG_INFO	Mensagens informativas
LOG_DEBUG	Mensagens de depuração (mais verboso)

4.27.3 Funções

4.27.3.1 log()

```
void log (
            LogLevel level,
            const char * fmt,
             ...)
```

Registra uma mensagem no log.

Exibe a mensagem apenas se o nível informado for maior ou igual ao nível global configurado em CURRENT_↩ LOG_LEVEL.

Parâmetros

level	Nível da mensagem (LOG_ERROR, LOG_WARN, LOG_INFO, LOG_DEBUG).
fmt	String formatada no estilo printf.
...	Argumentos variáveis para formatação.

Registra uma mensagem no log.

Esta função formata a mensagem recebida e imprime na Serial, respeitando o nível atual de log definido.

Parâmetros

level	Nível da mensagem (ERRO, WARN, INFO, DEBUG).
fmt	String de formato (similar ao printf).
...	Argumentos variáveis correspondentes ao formato.

4.27.3.2 logInit()

```
void logInit (
            LogLevel level)
```

Inicializa o sistema de log.

Configura o nível de log inicial e inicia a comunicação serial.

Parâmetros

level	Nível de log desejado (LOG_ERROR, LOG_WARN, LOG_INFO, LOG_DEBUG).

Inicializa o sistema de log.

- Define o nível de log inicial.
- Inicia a comunicação Serial a 115200 baud.
- Registra uma mensagem indicando que o log foi iniciado.

Parâmetros

level	Nível de log a ser utilizado (LOG_ERROR, LOG_WARN, LOG_INFO, LOG_DEBUG).

4.27.4 Variáveis

4.27.4.1 CURRENT_LOG_LEVEL

```
LogLevel CURRENT_LOG_LEVEL  [extern]
```

Variável global que armazena o nível de log atual.

Controla quais mensagens serão exibidas. Deve ser inicializada pela função logInit().

4.28 log.h

Ir para a documentação desse arquivo.

```
00001 #ifndef LOG_H
00002 #define LOG_H
00003
00004 #include <Arduino.h>
00005 #include <stdarg.h>
00006
00016
00021 enum LogLevel {
00022   LOG_NONE = 0,
00023   LOG_ERROR = 1,
00024   LOG_WARN = 2,
00025   LOG_INFO = 3,
00026   LOG_DEBUG = 4
00027 };
00028
00035 extern LogLevel CURRENT_LOG_LEVEL;
00036
00044 void logInit(LogLevel level);
00045
00056 void log(LogLevel level, const char* fmt, ...);
00057
00058 #endif
```

4.29 Referência do Arquivo microcontrolador_central/mqtt.cpp

Implementação do módulo de comunicação MQTT do sistema.

```
#include <WiFiClientSecure.h>
#include "mqtt.h"
#include "config.h"
#include "jsonDataProcessing.h"
#include "log.h"
#include "buzzer.h"
#include "config_storage.h"
```

Funções

- PubSubClient client (espClient)

  Cliente MQTT responsável pela comunicação com o broker.
- void resetMQTTClient ()

  Reconfigura o cliente MQTT e o cliente TLS (WiFiClientSecure).
- void setupMQTT ()

  Configura a conexão com o servidor MQTT.
- void checkMQTTConnected ()

  Verifica a conexão com o broker MQTT e reconecta se necessário.
- void callback (char ∗topic, byte ∗payload, unsigned int length)

  Callback executada quando uma mensagem MQTT é recebida.
- bool publicReponseDivice (const char ∗deviceId, const char ∗topic, const char ∗message, bool retained)

  Publica uma mensagem MQTT para um dispositivo específico.

Variáveis

- WiFiClient espClient

  Cliente de rede utilizado para comunicação com o broker MQTT.
- bool hasOKMessage = false

  Indica que uma mensagem de confirmação foi recebida do servidor MQTT.
- const char ∗ calledToBeErased

  Armazena o identificador da chamada que deve ser removida.

4.29.1 Descrição detalhada

Implementação do módulo de comunicação MQTT do sistema.

Este módulo é responsável por gerenciar toda a comunicação MQTT do dispositivo, incluindo:

- Conexão com o broker MQTT
- Reconexão automática em caso de falha
- Recebimento de mensagens através de callback
- Processamento de mensagens JSON recebidas
- Publicação de mensagens de confirmação

O módulo utiliza a biblioteca PubSubClient para comunicação MQTT e WiFiClientSecure para estabelecer conexões seguras via TLS/SSL.

Fluxo principal de funcionamento:

1. setupMQTT() configura o cliente MQTT e registra o callback.
2. checkMQTTConnected() mantém a conexão ativa e tenta reconectar se necessário.
3. Quando uma mensagem chega, a função callback() é executada.
4. O JSON recebido é processado por processing_json_MQTT().
5. Após processamento, o dispositivo envia uma confirmação ao broker.

Além disso, o módulo também:

- Ativa um alerta sonoro ao receber uma nova chamada.
- Controla o recebimento de confirmações do servidor.

Este módulo faz parte do sistema de chamadas de enfermagem baseado em MQTT.

4.29.2 Funções

4.29.2.1 callback()

```
void callback (
            char * topic,
            byte * payload,
            unsigned int length)
```

Callback executada quando uma mensagem MQTT é recebida.

Parâmetros

topic	Tópico no qual a mensagem foi publicada.
payload	Dados da mensagem recebida.
length	Tamanho do payload recebido.

Comportamento da função:

1. Se a mensagem for de confirmação:
   - Extrai o ID da chamada confirmada.
   - Sinaliza que essa chamada deve ser removida da lista.
2. Se a mensagem for uma nova chamada:
   - Processa o JSON recebido.
   - Envia uma confirmação ao servidor MQTT.
   - Ativa o alerta sonoro.

4.29.2.2 checkMQTTConnected()

```
void checkMQTTConnected ()
```

Verifica a conexão com o broker MQTT e reconecta se necessário.

Verifica e mantém a conexão com o broker MQTT.

Esta função deve ser chamada periodicamente dentro do loop principal do firmware.

Funções executadas:

- Mantém a comunicação MQTT ativa (client.loop()).
- Verifica se o cliente ainda está conectado.
- Caso a conexão tenha sido perdida, tenta reconectar após um intervalo.

4.29.2.3 client()

```
PubSubClient client (
            espClient )
```

Cliente MQTT responsável pela comunicação com o broker.

Utiliza o objeto espClient como interface de transporte para a rede. Esse cliente gerencia:

- conexão com o broker MQTT
- envio de mensagens
- recebimento de mensagens
- manutenção da sessão MQTT

4.29.2.4 publicReponseDivice()

```
bool publicReponseDivice (
            const char * deviceId,
            const char * topic,
            const char * message,
            bool retained)
```

Publica uma mensagem MQTT para um dispositivo específico.

Publica uma mensagem em um tópico MQTT específico do dispositivo.

O tópico final é formado concatenando: tópico_base + "/" + deviceId

Parâmetros

deviceId	ID do dispositivo associado à mensagem.
topic	Tópico base de publicação.
message	Conteúdo da mensagem a ser enviada.
retained	Define se a mensagem deve ser mantida no broker.

Retorna

true se a mensagem foi publicada com sucesso.

false se ocorreu falha na publicação.

4.29.2.5 resetMQTTClient()

```
void resetMQTTClient ()
```

Reconfigura o cliente MQTT e o cliente TLS (WiFiClientSecure).

Essa função é chamada quando o cliente MQTT falha repetidamente ao tentar reconectar ao broker. Em situações assim, o cliente TLS pode ficar em um estado inválido, bloqueando novas conexões. O procedimento abaixo recria ambos os clientes para restaurar a comunicação.

Operações executadas:

- Reinstancia o objeto `WiFiClientSecure` (limpa conexões antigas);
- Redefine o servidor MQTT e a porta segura;
- Reassocia o cliente TLS ao cliente MQTT.

Essa rotina evita a necessidade de reiniciar o microcontrolador.

4.29.2.6 setupMQTT()

```
void setupMQTT ()
```

Configura a conexão com o servidor MQTT.

Inicializa e configura o cliente MQTT.

Define parâmetros básicos como servidor, porta e função de callback que será chamada ao receber mensagens.

4.29.3 Variáveis

4.29.3.1 calledToBeErased

```
const char* calledToBeErased
```

Armazena o identificador da chamada que deve ser removida.

< Biblioteca que implementa o cliente MQTT (conexão, publish, subscribe).

O valor é extraído da mensagem de confirmação recebida via MQTT. Após o processamento da confirmação, a chamada correspondente poderá ser removida da lista local de chamadas de enfermagem.

4.29.3.2 espClient

```
WiFiClient espClient
```

Cliente de rede utilizado para comunicação com o broker MQTT.

Este objeto gerencia a conexão TCP/IP através da rede Wi-Fi e é utilizado pelo cliente MQTT como camada de transporte para envio e recebimento de mensagens.

4.29.3.3 hasOKMessage

```
bool hasOKMessage = false
```

Indica que uma mensagem de confirmação foi recebida do servidor MQTT.

Indica que foi recebida uma mensagem de confirmação do servidor.

Quando o servidor confirma o processamento de uma chamada, essa flag é ativada para que o sistema possa tratar a confirmação e atualizar o estado interno da aplicação, deletando a chamada que foi atendida.

4.30 Referência do Arquivo microcontrolador_central/mqtt.h

Interface do módulo de comunicação MQTT do dispositivo.

```
#include <PubSubClient.h>
```

Funções

- void setupMQTT ()

  Inicializa e configura o cliente MQTT.
- void checkMQTTConnected ()

  Verifica e mantém a conexão com o broker MQTT.
- void callback (char ∗topic, byte ∗payload, unsigned int length)

  Callback executada quando uma mensagem MQTT é recebida.
- bool publicReponseDivice (const char ∗deviceId, const char ∗topic, const char ∗message, bool retained=false)

  Publica uma mensagem em um tópico MQTT específico do dispositivo.

Variáveis

- const char ∗ calledToBeErased

  < Biblioteca que implementa o cliente MQTT (conexão, publish, subscribe).
- bool hasOKMessage

  Indica que foi recebida uma mensagem de confirmação do servidor.
- PubSubClient client

  Cliente MQTT global utilizado pelo sistema.

4.30.1 Descrição detalhada

Interface do módulo de comunicação MQTT do dispositivo.

Este módulo fornece as funções e variáveis necessárias para gerenciar a comunicação MQTT do sistema, incluindo:

- configuração da conexão com o broker MQTT
- manutenção da conexão
- recebimento de mensagens através de callback
- publicação de mensagens no broker

O módulo utiliza a biblioteca PubSubClient para implementação do protocolo MQTT.

4.30.2 Funções

4.30.2.1 callback()

```
void callback (
            char * topic,
            byte * payload,
            unsigned int length)
```

Callback executada quando uma mensagem MQTT é recebida.

Esta função é chamada automaticamente pela biblioteca MQTT sempre que uma nova mensagem chega em um tópico inscrito.

Parâmetros

topic	Tópico onde a mensagem foi publicada.
payload	Ponteiro para os dados recebidos.
length	Tamanho do payload em bytes.
topic	Tópico no qual a mensagem foi publicada.
payload	Dados da mensagem recebida.
length	Tamanho do payload recebido.

Comportamento da função:

1. Se a mensagem for de confirmação:
   - Extrai o ID da chamada confirmada.
   - Sinaliza que essa chamada deve ser removida da lista.
2. Se a mensagem for uma nova chamada:
   - Processa o JSON recebido.
   - Envia uma confirmação ao servidor MQTT.
   - Ativa o alerta sonoro.

4.30.2.2 checkMQTTConnected()

```
void checkMQTTConnected ()
```

Verifica e mantém a conexão com o broker MQTT.

Essa função deve ser chamada periodicamente no loop principal do programa. Ela executa:

- manutenção da comunicação MQTT (client.loop())
- verificação do estado da conexão
- tentativa automática de reconexão caso a conexão seja perdida

Verifica e mantém a conexão com o broker MQTT.

Esta função deve ser chamada periodicamente dentro do loop principal do firmware.

Funções executadas:

- Mantém a comunicação MQTT ativa (client.loop()).
- Verifica se o cliente ainda está conectado.
- Caso a conexão tenha sido perdida, tenta reconectar após um intervalo.

4.30.2.3 publicReponseDivice()

```
bool publicReponseDivice (
            const char * deviceId,
            const char * topic,
            const char * message,
            bool retained)
```

Publica uma mensagem em um tópico MQTT específico do dispositivo.

O tópico final é montado dinamicamente concatenando:

`topic_base/deviceId`

Parâmetros

deviceId	Identificador do dispositivo associado à mensagem.
topic	Tópico base de publicação.
message	Conteúdo da mensagem a ser enviada.
retained	Define se a mensagem deve ser mantida no broker.

Retorna

true se a publicação foi realizada com sucesso.

false caso ocorra falha no envio da mensagem.

Publica uma mensagem em um tópico MQTT específico do dispositivo.

O tópico final é formado concatenando: tópico_base + "/" + deviceId

Parâmetros

deviceId	ID do dispositivo associado à mensagem.
topic	Tópico base de publicação.
message	Conteúdo da mensagem a ser enviada.
retained	Define se a mensagem deve ser mantida no broker.

Retorna

true se a mensagem foi publicada com sucesso.

false se ocorreu falha na publicação.

4.30.2.4 setupMQTT()

```
void setupMQTT ()
```

Inicializa e configura o cliente MQTT.

Define o servidor MQTT, a porta de comunicação e registra a função callback responsável por tratar mensagens recebidas.

Esta função deve ser chamada durante a inicialização do sistema.

Inicializa e configura o cliente MQTT.

Define parâmetros básicos como servidor, porta e função de callback que será chamada ao receber mensagens.

4.30.3 Variáveis

4.30.3.1 calledToBeErased

```
const char* calledToBeErased [extern]
```

< Biblioteca que implementa o cliente MQTT (conexão, publish, subscribe).

ID da chamada que deve ser removida após confirmação do servidor.

Essa variável é atualizada quando uma mensagem de confirmação é recebida do broker MQTT. O valor representa o identificador da chamada que deve ser removida da lista local do dispositivo.

< Biblioteca que implementa o cliente MQTT (conexão, publish, subscribe).

O valor é extraído da mensagem de confirmação recebida via MQTT. Após o processamento da confirmação, a chamada correspondente poderá ser removida da lista local de chamadas de enfermagem.

4.30.3.2 client

```
PubSubClient client [extern]
```

Cliente MQTT global utilizado pelo sistema.

Esse objeto gerencia:

- conexão com o broker
- envio de mensagens (publish)
- recebimento de mensagens (subscribe)
- manutenção da sessão MQTT

A definição do objeto está localizada no arquivo `mqtt.cpp`.

4.30.3.3 hasOKMessage

```
bool hasOKMessage [extern]
```

Indica que foi recebida uma mensagem de confirmação do servidor.

Quando uma confirmação é recebida via MQTT, essa flag é ativada para que o sistema possa processar a remoção da chamada correspondente.

Indica que foi recebida uma mensagem de confirmação do servidor.

Quando o servidor confirma o processamento de uma chamada, essa flag é ativada para que o sistema possa tratar a confirmação e atualizar o estado interno da aplicação, deletando a chamada que foi atendida.

4.31 mqtt.h

Ir para a documentação desse arquivo.

```
#ifndef MQTT_HELPER_H
#define MQTT_HELPER_H


#include <PubSubClient.h>

// ---------------------------------------------------------------------------
// Declarações globais
// ---------------------------------------------------------------------------

extern const char* calledToBeErased;

extern bool hasOKMessage;

extern PubSubClient client;


// ---------------------------------------------------------------------------
// Assinaturas das funções
// ---------------------------------------------------------------------------

void setupMQTT();

void checkMQTTConnected();

void callback(char* topic, byte* payload, unsigned int length);

bool publicReponseDivice(const char* deviceId, const char* topic, const char* message, bool retained =
false);

#endif // MQTT_HELPER_H
```

4.32 Referência do Arquivo microcontrolador_central/server.cpp

Implementação do servidor HTTP para configuração do dispositivo.

```
#include "server.h"
#include "Arduino.h"
#include "log.h"
#include <ESP8266WiFi.h>
#include "config_storage.h"
```

Funções

- void serverSetupRoutes (ESP8266WebServer *server)

  Configura as rotas e endpoints do servidor web.
- void startServer (ESP8266WebServer *server)

  Inicia o servidor HTTP e registra suas rotas.
- void stopServer (ESP8266WebServer *server)

  Encerra o servidor HTTP e desativa o Access Point.
- void server_handle_loop (ESP8266WebServer *server)

  Mantém o servidor HTTP em execução.

Variáveis

- bool serverRunning = false

  Indica se o servidor HTTP está em execução.

4.32.1 Descrição detalhada

Implementação do servidor HTTP para configuração do dispositivo.

Este módulo implementa um servidor HTTP embarcado utilizado para configuração inicial do sistema através de uma interface web.

O servidor permite que o usuário configure parâmetros essenciais do dispositivo diretamente pelo navegador, sem necessidade de reprogramar o firmware.

As principais funcionalidades incluem:

- Exibir uma página web de configuração do sistema
- Receber parâmetros de configuração via formulário HTML
- Armazenar configurações em memória persistente
- Reiniciar o dispositivo após salvar novas configurações

4.32.1.1 Parâmetros configuráveis

A interface web permite configurar:

4.32.1.1.1 Wi-Fi

- SSID da rede
- Senha da rede

4.32.1.1.2 MQTT

- Endereço do broker MQTT
- Usuário
- Senha
- Device ID do dispositivo
- Porta do broker

4.32.1.2 Fluxo de funcionamento

1. O dispositivo inicia em modo **Access Point**.
2. O usuário conecta-se ao AP e acessa a página web do dispositivo.
3. O servidor exibe um formulário de configuração.
4. O usuário envia os parâmetros através do formulário.
5. O sistema salva as configurações na memória persistente.
6. O dispositivo reinicia para aplicar as novas configurações.

4.32.1.3 Dependências

Este módulo utiliza:

- ESP8266WebServer (servidor HTTP)
- ESP8266WiFi (controle da interface Wi-Fi)
- config_storage (armazenamento das configurações)

4.32.1.4 Arquitetura

As principais funções deste módulo são:

- startServer() → inicia o servidor HTTP
- stopServer() → encerra o servidor
- server_handle_loop() → processa requisições HTTP
- serverSetupRoutes() → define os endpoints da aplicação

Observação

O servidor é utilizado apenas para configuração do dispositivo e normalmente é executado quando o sistema não possui configurações válidas ou quando entra em modo de configuração.

4.32.2 Funções

4.32.2.1 server_handle_loop()

```
void server_handle_loop (
            ESP8266WebServer * server)
```

Mantém o servidor HTTP em execução.

Mantém o servidor ativo e processa requisições de clientes.

Deve ser chamada dentro do `loop()` principal para processar requisições de clientes conectados.

Parâmetros

server	Ponteiro para a instância de `ESP8266WebServer`.

4.32.2.2 serverSetupRoutes()

```
void serverSetupRoutes (
            ESP8266WebServer * server)
```

Configura as rotas e endpoints do servidor web.

Esta função registra os caminhos (URLs) que o servidor HTTP irá tratar:

- **"/"**: Exibe uma página HTML de configuração do sistema, permitindo ao usuário definir parâmetros de Wi-Fi, MQTT e limites ambientais.
- **"/submit"**: Recebe os dados enviados pelo formulário e salva as configurações na memória persistente.

Parâmetros

server	Ponteiro para a instância do servidor HTTP (`ESP8266WebServer`).

4.32.2.3 startServer()

```
void startServer (
            ESP8266WebServer * server)
```

Inicia o servidor HTTP e registra suas rotas.

Inicia o servidor HTTP e configura as rotas de acesso.

Esta função garante que o servidor só será iniciado uma vez. Chama `serverSetupRoutes()` para registrar os endpoints e inicia o servidor com `server->begin()`.

Parâmetros

server	Ponteiro para a instância de `ESP8266WebServer`.

4.32.2.4 stopServer()

```
void stopServer (
            ESP8266WebServer * server)
```

Encerra o servidor HTTP e desativa o Access Point.

Encerra o servidor HTTP e desconecta o Access Point.

Esta função interrompe conexões ativas, encerra o serviço HTTP e retorna o módulo Wi-Fi para o modo estação (**WIFI_STA**).

Parâmetros

server	Ponteiro para a instância de `ESP8266WebServer`.

4.32.3 Variáveis

4.32.3.1 serverRunning

```
bool serverRunning = false
```

Indica se o servidor HTTP está em execução.

Esta flag evita que o servidor seja iniciado múltiplas vezes. É configurada para `true` após a inicialização bem-sucedida e retornada a `false` quando o servidor é parado.

4.33 Referência do Arquivo microcontrolador_central/server.h

Interface do módulo de servidor HTTP para configuração do dispositivo.

```
#include <ESP8266WebServer.h>
```

Funções

- void startServer (ESP8266WebServer *server)

  Inicia o servidor HTTP e configura as rotas de acesso.
- void stopServer (ESP8266WebServer *server)

  Encerra o servidor HTTP e desconecta o Access Point.
- void server_setup_routes (ESP8266WebServer *server)

  Configura as rotas HTTP do servidor.
- void server_handle_loop (ESP8266WebServer *server)

  Mantém o servidor ativo e processa requisições de clientes.

4.33.1 Descrição detalhada

Interface do módulo de servidor HTTP para configuração do dispositivo.

Este módulo define a interface do servidor HTTP embarcado utilizado para configurar o dispositivo através de uma interface web.

O servidor permite que o usuário configure parâmetros essenciais do sistema, como credenciais de rede Wi-Fi e informações de conexão com o broker MQTT, diretamente por um navegador.

4.33.1.1 Funcionalidades principais

- Inicialização do servidor HTTP
- Configuração das rotas de acesso (endpoints)
- Processamento de requisições HTTP
- Encerramento do servidor e limpeza de recursos

4.33.1.2 Fluxo de uso

Normalmente o servidor é utilizado da seguinte forma:

1. Criar uma instância de `ESP8266WebServer`.
2. Iniciar o servidor utilizando `startServer()`.
3. No `loop()` principal, chamar `server_handle_loop()` para processar requisições HTTP.
4. Quando necessário, parar o servidor com `stopServer()`.

4.33.1.3 Dependências

Este módulo utiliza a biblioteca:

- ESP8266WebServer

responsável por implementar o servidor HTTP no ESP8266.

Observação

O servidor é geralmente utilizado no modo de configuração do dispositivo, permitindo que o usuário forneça parâmetros necessários para o funcionamento do sistema.

4.33.2 Funções

4.33.2.1 server_handle_loop()

```
void server_handle_loop (
            ESP8266WebServer * server)
```

Mantém o servidor ativo e processa requisições de clientes.

Essa função deve ser chamada no loop principal do programa (`loop()`), garantindo que o servidor trate novas conexões e requisições HTTP.

Parâmetros

server	Ponteiro para a instância de ESP8266WebServer em execução.

Mantém o servidor ativo e processa requisições de clientes.

Deve ser chamada dentro do `loop()` principal para processar requisições de clientes conectados.

Parâmetros

server	Ponteiro para a instância de `ESP8266WebServer`.

4.33.2.2 server_setup_routes()

```
void server_setup_routes (
            ESP8266WebServer * server)
```

Configura as rotas HTTP do servidor.

Define as páginas e endpoints que o servidor responderá. Normalmente, inclui a rota raiz ("/") e a rota de envio de formulário ("/submit").

Parâmetros

server	Ponteiro para a instância de ESP8266WebServer onde as rotas serão registradas.

4.33.2.3 startServer()

```
void startServer (
            ESP8266WebServer * server)
```

Inicia o servidor HTTP e configura as rotas de acesso.

Essa função chama as rotas definidas em server_setup_routes() e inicia o serviço do servidor. Deve ser chamada após configurar o modo Access Point (AP) ou conectar o dispositivo a uma rede Wi-Fi.

Parâmetros

server	Ponteiro para a instância de ESP8266WebServer a ser iniciada.

Inicia o servidor HTTP e configura as rotas de acesso.

Esta função garante que o servidor só será iniciado uma vez. Chama `serverSetupRoutes()` para registrar os endpoints e inicia o servidor com `server->begin()`.

Parâmetros

server	Ponteiro para a instância de `ESP8266WebServer`.

4.33.2.4 stopServer()

```
void stopServer (
            ESP8266WebServer * server)
```

Encerra o servidor HTTP e desconecta o Access Point.

Essa função interrompe o atendimento de requisições HTTP, encerra a conexão com os clientes e desativa o modo AP do Wi-Fi.

Parâmetros

server	Ponteiro para a instância de ESP8266WebServer a ser parada.

Encerra o servidor HTTP e desconecta o Access Point.

Esta função interrompe conexões ativas, encerra o serviço HTTP e retorna o módulo Wi-Fi para o modo estação (**WIFI_STA**).

Parâmetros

server	Ponteiro para a instância de `ESP8266WebServer`.

4.34 server.h

Ir para a documentação desse arquivo.

```
00001
00043
00044 #ifndef SERVER_H
00045 #define SERVER_H
00046
00047 #include <ESP8266WebServer.h>  // Biblioteca para criar e gerenciar um servidor HTTP no ESP8266
00048
00058 void startServer(ESP8266WebServer *server);
00059
00068 void stopServer(ESP8266WebServer *server);
00069
00078 void server_setup_routes(ESP8266WebServer* server);
00079
00088 void server_handle_loop(ESP8266WebServer* server);
00089
00090 #endif
```

4.35 Referência do Arquivo microcontrolador_central/wifi_utils.cpp

Implementação do módulo de gerenciamento de conexão Wi-Fi do dispositivo.

```
#include <ESP8266WiFi.h>
#include "wifi_utils.h"
#include "config.h"
#include "log.h"
#include "config_storage.h"
#include "display_LCD-2004_I2C.h"
```

Funções

- const char * IPparserToConstChar (IPAddress ip)

  Converte um objeto IPAddress para uma string constante.
- bool connectToWiFi ()

  Conecta o dispositivo à rede Wi-Fi configurada.
- bool checkAndReconnectWifi ()

  Verifica o estado da conexão Wi-Fi e tenta reconectar se necessário.
- void createAccessPoint ()

  Cria um Access Point para configuração do dispositivo.
- bool wifiIsConnected ()

  Retorna se o ESP32 está conectado ao Wi-Fi.

Variáveis

- char ipBuffer [16]

  Buffer utilizado para armazenar o endereço IP em formato string.

4.35.1 Descrição detalhada

Implementação do módulo de gerenciamento de conexão Wi-Fi do dispositivo.

Este módulo é responsável por controlar toda a conectividade Wi-Fi do sistema. Ele fornece funções para:

- conectar o dispositivo a uma rede Wi-Fi existente
- monitorar o estado da conexão
- realizar reconexão automática quando necessário
- criar um Access Point para configuração do dispositivo

O módulo também exibe o endereço IP obtido no display LCD do sistema quando a conexão é estabelecida.

4.35.1.1 Fluxo de funcionamento

1. O sistema tenta conectar à rede Wi-Fi utilizando `connectToWiFi()`.
2. Durante a execução normal, `checkAndReconnectWifi()` monitora a conexão.
3. Caso a conexão seja perdida, o sistema tenta reconectar automaticamente.
4. Se não houver rede disponível ou o dispositivo estiver em modo de configuração, `createAccessPoint()` cria um ponto de acesso local.

4.35.1.2 Dependências

Este módulo utiliza:

- ESP8266WiFi → gerenciamento da interface Wi-Fi
- config_storage → leitura das configurações de rede
- log → registro de eventos do sistema
- display_LCD-2004_I2C → exibição do endereço IP no display

4.35.2 Funções

4.35.2.1 checkAndReconnectWifi()

```
bool checkAndReconnectWifi ()
```

Verifica o estado da conexão Wi-Fi e tenta reconectar se necessário.

Verifica o estado da conexão Wi-Fi e tenta reconectar.

Esta função deve ser chamada periodicamente dentro do `loop()` principal do firmware.

Caso o dispositivo esteja desconectado da rede, uma nova tentativa de conexão será realizada respeitando o intervalo definido em `reconnectInterval`.

Retorna

true se o dispositivo estiver conectado ao Wi-Fi.

false caso continue desconectado.

4.35.2.2 connectToWiFi()

```
bool connectToWiFi ()
```

Conecta o dispositivo à rede Wi-Fi configurada.

A função utiliza as credenciais armazenadas em `cfg.wifiSSID` e `cfg.wifiPass`.

O sistema aguarda um tempo limitado pela conexão e exibe o endereço IP obtido no display LCD caso a conexão seja bem sucedida.

Retorna

true se a conexão foi estabelecida com sucesso.

false caso a conexão falhe ou expire o tempo limite.

4.35.2.3 createAccessPoint()

```
void createAccessPoint ()
```

Cria um Access Point para configuração do dispositivo.

Caso o dispositivo não esteja operando em modo Access Point, esta função:

- Desconecta a interface Wi-Fi atual
- Define o modo `WIFI_AP`
- Cria um Access Point com SSID e senha definidos nas constantes `SSID_ACCESS_POINT` e `PASSWORD_ACCESS_POINT`

O endereço IP do Access Point também é exibido no display LCD.

4.35.2.4 IPparserToConstChar()

```
const char * IPparserToConstChar (
            IPAddress ip)
```

Converte um objeto IPAddress para uma string constante.

Parâmetros

ip	Endereço IP a ser convertido.

Retorna

Ponteiro para string contendo o endereço IP no formato `"xxx.xxx.xxx.xxx"`.

4.35.2.5 wifiIsConnected()

```
bool wifiIsConnected ()
```

Retorna se o ESP32 está conectado ao Wi-Fi.

Verifica se o dispositivo está conectado à rede Wi-Fi.

Retorna

true se conectado, false caso contrário.

4.35.3 Variáveis

4.35.3.1 ipBuffer

```
char ipBuffer[16]
```

Buffer utilizado para armazenar o endereço IP em formato string.

Exemplo de valor armazenado: `"192.168.1.10"`

4.36 Referência do Arquivo microcontrolador_central/wifi_utils.h

Interface do módulo de gerenciamento de conexão Wi-Fi.

```
#include <ESP8266WiFi.h>
#include "config.h"
```

Funções

- bool connectToWiFi ()

  Conecta o dispositivo à rede Wi-Fi configurada.
- bool checkAndReconnectWifi ()

  Verifica o estado da conexão Wi-Fi e tenta reconectar.
- void createAccessPoint ()

  Cria um Access Point para configuração do dispositivo.
- bool wifiIsConnected ()

  Verifica se o dispositivo está conectado à rede Wi-Fi.
- const char ∗ **getIPwifi** ()

4.36.1 Descrição detalhada

Interface do módulo de gerenciamento de conexão Wi-Fi.

Este módulo fornece funções para controle da conectividade Wi-Fi do dispositivo. Ele permite:

- Conectar o dispositivo a uma rede Wi-Fi configurada
- Monitorar o estado da conexão
- Realizar reconexão automática em caso de perda de conexão
- Criar um Access Point para configuração local

Essas funções são utilizadas pelo firmware principal para garantir que o dispositivo permaneça conectado à rede e possa se comunicar com outros serviços (como MQTT).

4.36.1.1 Fluxo típico de uso

1. O sistema tenta conectar à rede com `connectToWiFi()`.
2. Durante a execução normal, `checkAndReconnectWifi()` é chamada no `loop()` principal para verificar a conexão.
3. Caso não seja possível conectar à rede, o sistema pode criar um Access Point usando `createAccessPoint()` para permitir configuração local do dispositivo.

Observação

Este módulo depende da biblioteca `ESP8266WiFi`.

4.36.2 Funções

4.36.2.1 checkAndReconnectWifi()

```
bool checkAndReconnectWifi ()
```

Verifica o estado da conexão Wi-Fi e tenta reconectar.

Caso o dispositivo esteja desconectado da rede, a função aguarda o intervalo configurado internamente antes de tentar uma nova conexão.

Essa função deve ser chamada periodicamente no `loop()` principal.

Retorna

true se o dispositivo estiver conectado ao Wi-Fi.

false se continuar desconectado.

Verifica o estado da conexão Wi-Fi e tenta reconectar.

Esta função deve ser chamada periodicamente dentro do `loop()` principal do firmware.

Caso o dispositivo esteja desconectado da rede, uma nova tentativa de conexão será realizada respeitando o intervalo definido em `reconnectInterval`.

Retorna

true se o dispositivo estiver conectado ao Wi-Fi.

false caso continue desconectado.

4.36.2.2 connectToWiFi()

```
bool connectToWiFi ()
```

Conecta o dispositivo à rede Wi-Fi configurada.

Utiliza as credenciais armazenadas na estrutura de configuração (`cfg.wifiSSID` e `cfg.wifiPass`).

A função aguarda um tempo limitado pela conexão e retorna o resultado da tentativa.

Retorna

true se a conexão foi estabelecida com sucesso.

false se a conexão falhar.

A função utiliza as credenciais armazenadas em `cfg.wifiSSID` e `cfg.wifiPass`.

O sistema aguarda um tempo limitado pela conexão e exibe o endereço IP obtido no display LCD caso a conexão seja bem sucedida.

Retorna

true se a conexão foi estabelecida com sucesso.

false caso a conexão falhe ou expire o tempo limite.

4.36.2.3 createAccessPoint()

```
void createAccessPoint ()
```

Cria um Access Point para configuração do dispositivo.

Esta função coloca o dispositivo em modo `WIFI_AP` e cria um ponto de acesso utilizando o SSID e senha definidos nas constantes do sistema.

O endereço IP do Access Point pode ser utilizado pelo usuário para acessar a interface web de configuração.

Caso o dispositivo não esteja operando em modo Access Point, esta função:

- Desconecta a interface Wi-Fi atual
- Define o modo `WIFI_AP`
- Cria um Access Point com SSID e senha definidos nas constantes `SSID_ACCESS_POINT` e `PASSWORD_ACCESS_POINT`

O endereço IP do Access Point também é exibido no display LCD.

4.36.2.4 wifiIsConnected()

```
bool wifiIsConnected ()
```

Verifica se o dispositivo está conectado à rede Wi-Fi.

Retorna

true se o dispositivo estiver conectado.

false caso contrário.

Verifica se o dispositivo está conectado à rede Wi-Fi.

Retorna

true se conectado, false caso contrário.

4.37 wifi_utils.h

Ir para a documentação desse arquivo.

```
00001
00030
00031 #ifndef WIFI_HELPER_H
00032 #define WIFI_HELPER_H
00033
00034 #include <ESP8266WiFi.h>
00035 #include "config.h"
00036
00037
00050 bool connectToWiFi();
00051
00064 bool checkAndReconnectWifi();
00065
00066
00067
00078 void createAccessPoint();
00079
00086 bool wifiIsConnected();
00087
00088
00089 const char* getIPwifi();
00090
00091 #endif
```

Capítulo 5

Exemplos

5.1 Exemplo

de JSON recebido via MQTT

{ "id": "Enfermagem1", "estado": "emergencia", "mensagem": "Ligar LED", "room_number": "9", "local": "↩ Enfermaria", "comando": "ligar" }

Índice Remissivo

~List_NursingCall
List_NursingCall, 9

add
List_NursingCall, 10

Button, 5
lastDebounceTime, 5
lastRawReading, 5
state, 6
button_delete
buttons.cpp, 20
buttons.h, 25
button_next
buttons.cpp, 20
buttons.h, 25
button_prev
buttons.cpp, 20
buttons.h, 25
buttonPressStartTime
buttons.cpp, 21
buttons.cpp
button_delete, 20
button_next, 20
button_prev, 20
buttonPressStartTime, 21
checkButton, 19
DEBOUNCE_DELAY, 21
enableButtonInterruptChange, 19
initButtons, 19
isButtonLongPressActivate, 21
isButtonPressed, 21
isConfigurationMode, 19
longPressButtonISR, 20
buttons.h
button_delete, 25
button_next, 25
button_prev, 25
checkButton, 23
enableButtonInterruptChange, 23
initButtons, 24
isConfigurationMode, 24
buzzer.cpp
buzzerInit, 27
doesHaveNotificationBuzzer, 27
enableSoundAlert, 27
isBuzzerNotificationEnabled, 28
lastBuzzerPlayTime, 28
numberAlertIssued, 28
toggleBuzzer, 27
buzzer.h
BUZZER_INTERVAL_MS, 31
buzzerInit, 29
doesHaveNotificationBuzzer, 29
enableSoundAlert, 30
toggleBuzzer, 30
BUZZER_INTERVAL_MS
buzzer.h, 31
buzzerInit
buzzer.cpp, 27
buzzer.h, 29

callback
mqtt.cpp, 79
mqtt.h, 83
calledToBeErased
mqtt.cpp, 81
mqtt.h, 85
cfg
config_storage.cpp, 38
config_storage.h, 43
checkAndReconnectWifi
wifi_utils.cpp, 95
wifi_utils.h, 98
checkButton
buttons.cpp, 19
buttons.h, 23
checkMQTTConnected
mqtt.cpp, 79
mqtt.h, 83
clear
List_NursingCall, 10
clearLine
display_LCD-2004_I2C.cpp, 45
client
mqtt.cpp, 79
mqtt.h, 85
config.h
LOG_MODE, 33
MQTT_PORT, 33
NUMBER_SOUND_ALERTS, 33
PASSWORD_ACCESS_POINT, 33
PIN_BUZZER, 33
PIN_LED, 34
SSID_ACCESS_POINT, 34
CONFIG_PATH
config_storage.cpp, 38
config_storage.cpp
cfg, 38

CONFIG_PATH, 38
hasConfigData, 36
initConfigStorage, 36
loadConfig, 37
saveConfigutionData, 37
config_storage.h
cfg, 43
hasConfigData, 40
initConfigStorage, 40
loadConfig, 41
saveConfigutionData, 41
ConfigurationData, 6
mqttDeviceId, 7
mqttPass, 7
mqttPort, 7
mqttServer, 7
mqttUser, 7
valid, 7
wifiPass, 7
wifiSSID, 7
connectToWiFi
wifi_utils.cpp, 95
wifi_utils.h, 98
createAccessPoint
wifi_utils.cpp, 95
wifi_utils.h, 98
createJsonPayload
jsonDataProcessing.cpp, 57
jsonDataProcessing.h, 60
creteJsonPayloadConfirmationMessage
jsonDataProcessing.cpp, 58
jsonDataProcessing.h, 61
CURRENT_LOG_LEVEL
log.cpp, 74
log.h, 77
currentScreen
display_LCD-2004_I2C.cpp, 48
display_LCD-2004_I2C.h, 53

DEBOUNCE_DELAY
buttons.cpp, 21
display_LCD-2004_I2C.cpp
clearLine, 45
currentScreen, 48
fixed_data, 45
I2C_ADDR, 48
lcd, 45
lcd2004_init, 46
showExclusionConfirm, 46
showFailureMessage, 46
showInfirmaryNumber, 47
showIPAddress, 47
display_LCD-2004_I2C.h
currentScreen, 53
DisplayState, 50
lcd2004_init, 50
MESSAGE_MQTT, 53
MESSAGE_WIFI, 53
SCREEN_EXCLUSION_CONFIRM, 50
SCREEN_FAILURE_MQTT, 50
SCREEN_FAILURE_WIFI, 50
SCREEN_IPADDRESS, 50
SCREEN_MAIN, 50
SCREEN_NONE, 50
showExclusionConfirm, 50
showFailureMessage, 51
showInfirmaryNumber, 52
showIPAddress, 52
DisplayState
display_LCD-2004_I2C.h, 50
doesHaveNotificationBuzzer
buzzer.cpp, 27
buzzer.h, 29

enableButtonInterruptChange
buttons.cpp, 19
buttons.h, 23
enableSoundAlert
buzzer.cpp, 27
buzzer.h, 30
espClient
mqtt.cpp, 81

fixed_data
display_LCD-2004_I2C.cpp, 45

getDoNotRemoveCurrent
List_NursingCall, 10
getIdCurrent
List_NursingCall, 11
getInfirmaryCurrent
List_NursingCall, 11
getPayloadID
jsonDataProcessing.cpp, 58
jsonDataProcessing.h, 62
getTotal
List_NursingCall, 11

hasConfigData
config_storage.cpp, 36
config_storage.h, 40
hasNursingCall
List_NursingCall, 11
hasOKMessage
mqtt.cpp, 81
mqtt.h, 85

I2C_ADDR
display_LCD-2004_I2C.cpp, 48
id
NursingCall, 15
infirmary
NursingCall, 15
initButtons
buttons.cpp, 19
buttons.h, 24
initConfigStorage
config_storage.cpp, 36

config_storage.h, 40
ipBuffer
wifi_utils.cpp, 96
IPparserToConstChar
wifi_utils.cpp, 95
isButtonLongPressActivate
buttons.cpp, 21
isButtonPressed
buttons.cpp, 21
isBuzzerNotificationEnabled
buzzer.cpp, 28
isConfigurationMode
buttons.cpp, 19
buttons.h, 24

jsonDataProcessing.cpp
createJsonPayload, 57
creteJsonPayloadConfirmationMessage, 58
getPayloadID, 58
processing_json_MQTT, 59
jsonDataProcessing.h
createJsonPayload, 60
creteJsonPayloadConfirmationMessage, 61
getPayloadID, 62
processing_json_MQTT, 63

lastBuzzerPlayTime
buzzer.cpp, 28
lastDebounceTime
Button, 5
lastRawReading
Button, 5
lastTimeLedTurnedOn
led.cpp, 66
lcd
display_LCD-2004_I2C.cpp, 45
lcd2004_init
display_LCD-2004_I2C.cpp, 46
display_LCD-2004_I2C.h, 50
led.cpp
lastTimeLedTurnedOn, 66
LED_INTERVAL_MS, 66
ledInit, 65
toggleLed, 65
turnOffLed, 66
turnOnLed, 66
led.h
ledInit, 68
toggleLed, 68
turnOffLed, 68
turnOnLed, 68
LED_INTERVAL_MS
led.cpp, 66
ledInit
led.cpp, 65
led.h, 68
List_NursingCall, 8
~List_NursingCall, 9
add, 10
clear, 10
getDoNotRemoveCurrent, 10
getIdCurrent, 11
getInfirmaryCurrent, 11
getTotal, 11
hasNursingCall, 11
List_NursingCall, 9
next, 12
prev, 12
removalById, 12
removeCurrent, 13
setDoNotRemoveCurrent, 13
listCalls
listNursingCall_utils.cpp, 70
listNursingCall_utils.h, 72
listNursingCall_utils.cpp
listCalls, 70
listUpdated, 70
listNursingCall_utils.h
listCalls, 72
listUpdated, 72
listUpdated
listNursingCall_utils.cpp, 70
listNursingCall_utils.h, 72
loadConfig
config_storage.cpp, 37
config_storage.h, 41
log
log.cpp, 74
log.h, 76
log.cpp
CURRENT_LOG_LEVEL, 74
log, 74
logInit, 74
log.h
CURRENT_LOG_LEVEL, 77
log, 76
LOG_DEBUG, 76
LOG_ERROR, 75
LOG_INFO, 76
LOG_NONE, 75
LOG_WARN, 76
logInit, 76
LogLevel, 75
LOG_DEBUG
log.h, 76
LOG_ERROR
log.h, 75
LOG_INFO
log.h, 76
LOG_MODE
config.h, 33
LOG_NONE
log.h, 75
LOG_WARN
log.h, 76
logInit
log.cpp, 74

log.h, 76
LogLevel
log.h, 75
longPressButtonISR
buttons.cpp, 20

MESSAGE_MQTT
display_LCD-2004_I2C.h, 53
MESSAGE_WIFI
display_LCD-2004_I2C.h, 53
microcontrolador_central/buttons.cpp, 17
microcontrolador_central/buttons.h, 21, 25
microcontrolador_central/buzzer.cpp, 26
microcontrolador_central/buzzer.h, 28, 31
microcontrolador_central/config.h, 31, 34
microcontrolador_central/config_storage.cpp, 35
microcontrolador_central/config_storage.h, 39, 43
microcontrolador_central/display_LCD-2004_I2C.cpp, 44
microcontrolador_central/display_LCD-2004_I2C.h, 48, 54
microcontrolador_central/DoublyLinkedList_NursingCall.cpp, 54
microcontrolador_central/DoublyLinkedList_NursingCall.h, 55
microcontrolador_central/jsonDataProcessing.cpp, 56
microcontrolador_central/jsonDataProcessing.h, 59, 64
microcontrolador_central/led.cpp, 64
microcontrolador_central/led.h, 67, 69
microcontrolador_central/listNursingCall_utils.cpp, 69
microcontrolador_central/listNursingCall_utils.h, 70, 73
microcontrolador_central/log.cpp, 73
microcontrolador_central/log.h, 75, 77
microcontrolador_central/mqtt.cpp, 77
microcontrolador_central/mqtt.h, 82, 86
microcontrolador_central/server.cpp, 86
microcontrolador_central/server.h, 90, 93
microcontrolador_central/wifi_utils.cpp, 93
microcontrolador_central/wifi_utils.h, 96, 99
mqtt.cpp
callback, 79
calledToBeErased, 81
checkMQTTConnected, 79
client, 79
espClient, 81
hasOKMessage, 81
publicReponseDivice, 80
resetMQTTClient, 80
setupMQTT, 81
mqtt.h
callback, 83
calledToBeErased, 85
checkMQTTConnected, 83
client, 85
hasOKMessage, 85
publicReponseDivice, 83
setupMQTT, 84
MQTT_PORT
config.h, 33
mqttDeviceId
ConfigurationData, 7
mqttPass
ConfigurationData, 7
mqttPort
ConfigurationData, 7
mqttServer
ConfigurationData, 7
mqttUser
ConfigurationData, 7

next
List_NursingCall, 12
NursingCall, 15
NUMBER_SOUND_ALERTS
config.h, 33
numberAlertIssued
buzzer.cpp, 28
NursingCall, 14
id, 15
infirmary, 15
next, 15
prev, 15

PASSWORD_ACCESS_POINT
config.h, 33
PIN_BUZZER
config.h, 33
PIN_LED
config.h, 34
prev
List_NursingCall, 12
NursingCall, 15
processing_json_MQTT
jsonDataProcessing.cpp, 59
jsonDataProcessing.h, 63
publicReponseDivice
mqtt.cpp, 80
mqtt.h, 83

removalById
List_NursingCall, 12
removeCurrent
List_NursingCall, 13
resetMQTTClient
mqtt.cpp, 80

saveConfigutionData
config_storage.cpp, 37
config_storage.h, 41
SCREEN_EXCLUSION_CONFIRM
display_LCD-2004_I2C.h, 50
SCREEN_FAILURE_MQTT
display_LCD-2004_I2C.h, 50
SCREEN_FAILURE_WIFI
display_LCD-2004_I2C.h, 50
SCREEN_IPADDRESS
display_LCD-2004_I2C.h, 50
SCREEN_MAIN

display_LCD-2004_I2C.h, 50
SCREEN_NONE
display_LCD-2004_I2C.h, 50
server.cpp
server_handle_loop, 88
serverRunning, 90
serverSetupRoutes, 88
startServer, 89
stopServer, 89
server.h
server_handle_loop, 91
server_setup_routes, 91
startServer, 92
stopServer, 92
server_handle_loop
server.cpp, 88
server.h, 91
server_setup_routes
server.h, 91
serverRunning
server.cpp, 90
serverSetupRoutes
server.cpp, 88
setDoNotRemoveCurrent
List_NursingCall, 13
setupMQTT
mqtt.cpp, 81
mqtt.h, 84
showExclusionConfirm
display_LCD-2004_I2C.cpp, 46
display_LCD-2004_I2C.h, 50
showFailureMessage
display_LCD-2004_I2C.cpp, 46
display_LCD-2004_I2C.h, 51
showInfirmaryNumber
display_LCD-2004_I2C.cpp, 47
display_LCD-2004_I2C.h, 52
showIPAddress
display_LCD-2004_I2C.cpp, 47
display_LCD-2004_I2C.h, 52
SSID_ACCESS_POINT
config.h, 34
startServer
server.cpp, 89
server.h, 92
state
Button, 6
stopServer
server.cpp, 89
server.h, 92

toggleBuzzer
buzzer.cpp, 27
buzzer.h, 30
toggleLed
led.cpp, 65
led.h, 68
turnOffLed
led.cpp, 66
led.h, 68
turnOnLed
led.cpp, 66
led.h, 68

valid
ConfigurationData, 7

wifi_utils.cpp
checkAndReconnectWifi, 95
connectToWiFi, 95
createAccessPoint, 95
ipBuffer, 96
IPparserToConstChar, 95
wifiIsConnected, 96
wifi_utils.h
checkAndReconnectWifi, 98
connectToWiFi, 98
createAccessPoint, 98
wifiIsConnected, 99
wifiIsConnected
wifi_utils.cpp, 96
wifi_utils.h, 99
wifiPass
ConfigurationData, 7
wifiSSID
ConfigurationData, 7